Solving Berth Allocation Problems via Stable Sets

Alberto Santini
EMAIL: alberto.santini@upf.edu ORCID: 0000-0002-0440-0357

Department of Economics and Business, Universitat Pompeu Fabra, Barcelona, 08005 Spain

November 11, 2026

Abstract

Berth Allocation Problems (BAPs) are a family of well-known problems in port operations. They involve assigning incoming container ships to port berths to minimise one of several possible objective functions. In the literature, we identify *bottleneck* objective functions, whose value is often determined by a single ship and *cumulative* objective functions, whose value depends on all ships. For example, the makespan, i.e., the end-of-service time of the last ship, is a bottleneck objective, whereas the sum of the waiting and handling times of all ships is a cumulative objective. In this paper, we propose transformations of BAPs into stable set problems. In particular, we show that, for a large family of cumulative functions, the BAP can be solved with a single instance of the Maximum Weighted Stable Set Problem. Conversely, for a large family of bottleneck functions, the optimal solution of the BAP can be found by repeatedly solving the unweighted Maximum Stable Set Problem. We compare our proposed methods with five mixed-integer programming formulations from the literature that can be adapted for all problem variants. We report computational results on benchmark instances derived from 2024 data from the port of Barcelona. For bottleneck objectives, our method finds more optimal solutions and achieves lower average gaps within a one-hour time limit. For cumulative objectives and berth-dependent handling times, our method finds more optimal solutions in shorter computing times, although a formulation based on the generalised set partitioning problem often yields smaller optimality gaps. For cumulative objectives and berth-independent handling times, an adapted time-index formulation performs best across all metrics.

Keywords: Berth allocation problem, maximum stable set problem, maritime logistics. **MSC Codes:** 90B06.

1 Introduction

Berth Allocation Problems (BAPs) are a family of operational problems arising in port container terminal management. In a popular version, the port quay is divided into discrete berths, and incoming container ships must dock at available berths to load and unload their cargo. Each ship can require one or more berths for a prespecified amount of time, depending on its size. If no suitable berth is available when a ship reaches the port, it must wait. Figure 1 shows a quay divided into five berths and a moored ship occupying two. Port managers must decide when and where each ship can berth to minimise a given objective function, such as the total makespan (i.e., the completion time of the last served ship) or the sum of the waiting and handling times.

Different BAPs make different assumptions about how the port operates and the data available to the decision-maker. Bierwirth et al. (2015) proposes a classification along four attributes: spatial, temporal, handling time, and objective function.

Regarding the spatial attribute, problems belong to one of three classes. In the first, the quay is continuous and ships can berth at any point. In the second, it is divided into berths, and each ship occupies a single berth. In the third, the quay is divided into berths, and depending on its size, a ship can occupy more than one berth, or multiple ships can occupy a single berth. Our work mainly considers the third class ("hybr" in the classification). While our approach can also solve problems falling in the second class ("disc") as a special case, it does not explicitly exploit the limitation that

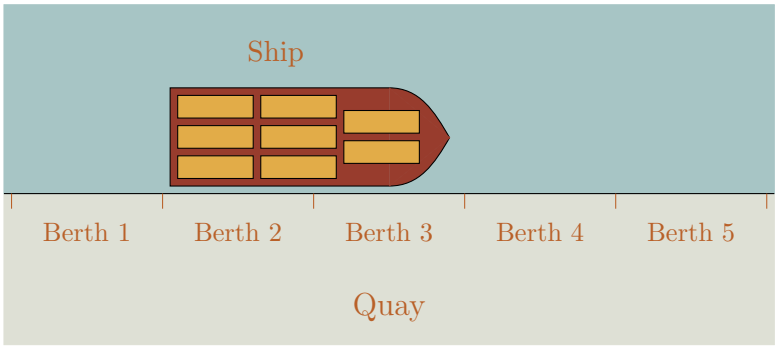


Figure 1: A quay divided into five berths, and a container ship occupying two of them.

each ship can only occupy one berth.

Concerning the temporal attribute, the two main classes of deterministic problems are: static, if all ships are assumed to be ready to berth at the beginning of the time horizon and dynamic, if ships have individual arrival times. We consider the dynamic case ("dyn" in the classification), although our approach also works in the static case ("stat").

The third attribute is the handling time. It can either depend on the ship ("fix" in the classification), on the ship and the assigned berth ("pos" in the classification), or on more complex decisions such as the number of assigned cranes (see, e.g., Vacca et al. 2012; Santini et al. 2015; Cahyono et al. 2022; Li et al. 2022; Ji et al. 2023). Our proposed approach can solve problems in the "fix" and "pos" classes.

The fourth attribute defines the objective function to minimise. In this work, we group the objective functions into two large groups: bottleneck and cumulative. The optimal solution of a problem with a bottleneck objective function depends on berthing decisions for a small number of ships (often just one). For example, two such objective functions used in the literature involve minimising either the makespan, "max(compl)" in the classification, or the total quay length required to handle all ships within a given deadline, "max(pos)". In these cases, it's, respectively, the ship that completes last and the one that uses the farthest berth in the quay that determines the value of the objective function. A small change in the berthing decisions of other ships is unlikely to affect the optimal value of the problem. Conversely, the optimal solution of a problem with a cumulative objective depends on the berthing decisions of all ships. Such objective functions include "$\sum$(wait+hand)" that minimises the total waiting and handling times and "$\sum$(wait+tard)" that includes tardiness. Changes to the berthing decisions of any ship are likely to change their corresponding waiting or handling times, or the tardiness and, therefore, they change the entire objective value.

In summary, according to the definition of Bierwirth et al. (2015), our proposed approaches are best suited for problems of type "hyb|stat,dyn|fix,pos|...", in which comma-separated classes indicate that the approach is valid for all the classes of a given attribute and three dots mean that a large class of objective functions are supported.

Figure 2 places our proposed approach in the existing literature on BAP variants. We present the variants hierarchically, distinguishing them by their objective function, and then by their spatial, temporal and handling attributes. Although our approach can solve several variants, among the bottleneck objectives, we focus on the max(compl) objective function rather than max(pos), because of its greater popularity and real-world relevance. Among the cumulative objectives, we consider all three objectives that have been identified in the literature, namely, $\sum$(compl), $\sum$w(wait+hand), and $\sum$w(wait+tard). We also focus on the hybrid spatial attribute because the resulting problem is more general than the discrete case and more realistically represents modern container terminals compared to the continuous case. We consider the dynamic temporal attribute because it generalises the static one. Finally, we consider problems with both position-based and fixed handling times.

After a comprehensive literature analysis, we identified five exact approaches which can be adapted to the problems we consider. These are: the relative position (RP) and position assignment (PA)

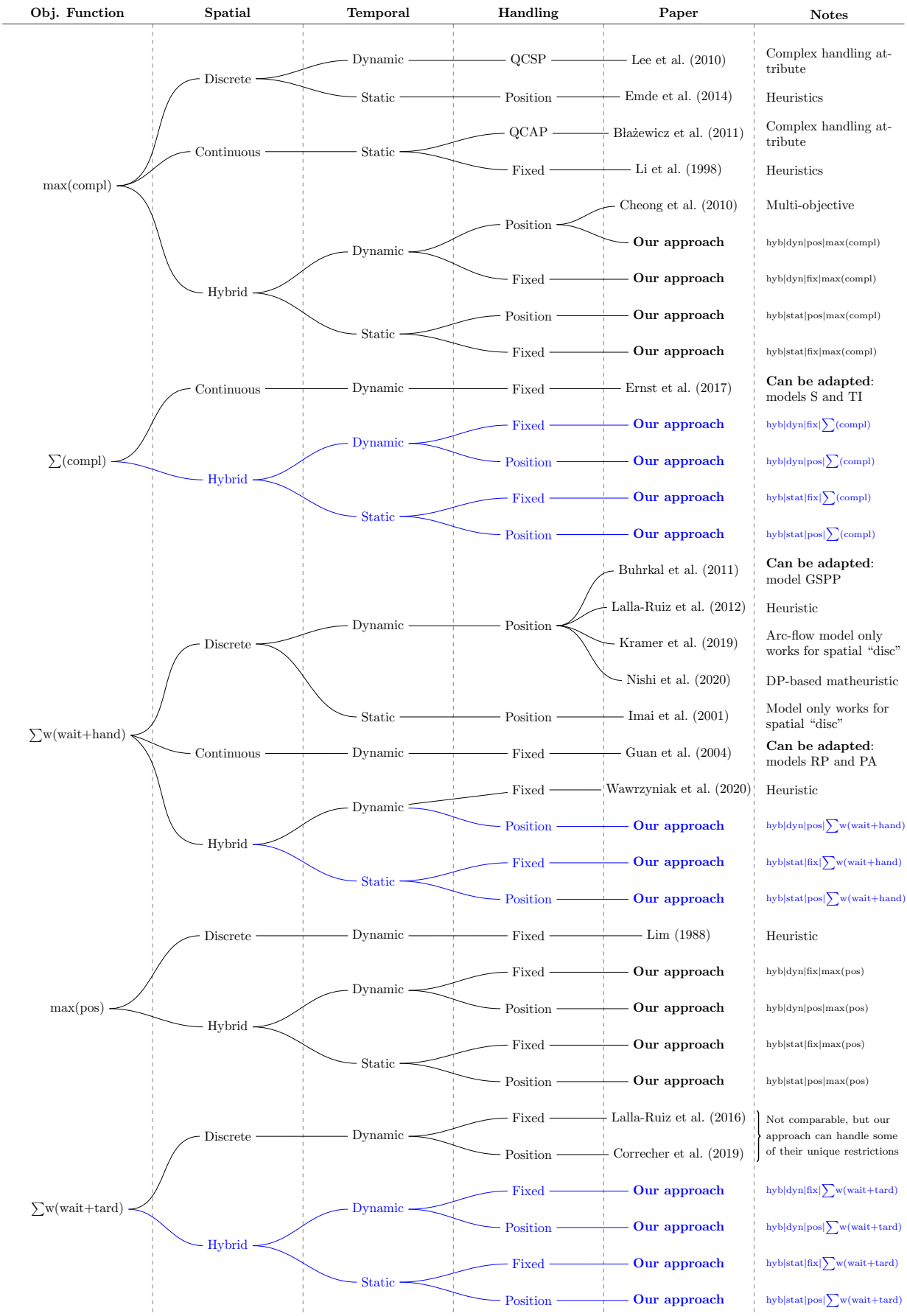


Figure 2: Positioning of our proposed methodology in the family of BAPs, with links to the relevant literature.

formulations of Guan et al. (2004); the generalised set partitioning problem (GSPP) formulation of Buhrkal et al. (2011); the sequence variables (S) and time index (TI) models proposed by Ernst et al. (2017). We discuss the existing literature further in Section 3.

Key contributions. The main contributions of this paper are:

- We give a formal proof of the equivalence between the decision version of the BAP with bottleneck objectives and the unweighted Maximum Stable Set Problem, and of the equivalence between the BAP with cumulateive objectives and the Maximum Weighted Stable Set Problem.
- Based on these properties, we propose effective solution approaches for both problem types and compare them with existing formulations from the literature.
- We introduce a new benchmark set based on real data from the port of Barcelona, Spain, which we make publicly available, together with the best known solution for each instance.
- Via an extensive computational campaign, we prove the effectiveness of our proposed approaches and of the existing formulations. We find that our new solution methods are particularly effective on BAPs with bottleneck objectives and on BAPs with cumulative objectives and berth-dependent handling times. A formulation based on the generalised set partitioning problem is useful to obtain strong dual bounds on BAPs with cumulative objectives and berth-dependent handling times, whereas a time-index formulation works best on BAPs with cumulative objectives and berth-independent handling times.

Paper organisation. The rest of this paper is organised as follows. In Section 2, we introduce mathematical notation common to the BAP variants, and we precisely describe the objective functions considered in this work. In Section 3, we place our contribution in the existing literature and identify five existing formulations that can be adapted to all the objective functions we consider. These formulations form a natural test bench to assess the performance of new exact methods for the BAPs. In Section 4, we present our approach for BAPs with bottleneck objective functions, based on a transformation to the unweighted Maximum Stable Set Problem, a bound-based search algorithm, and a dynamic time discretisation procedure. In Section 5, we introduce our approach for BAPs with cumulative objective functions, based on a transformation to the Maximum Weighted Stable Set Problem. To obtain effective solution methods, the proposed approaches are augmented with initial and improvement heuristics, graph pruning techniques, bounding procedures and other techniques described in Section 6. Section 7 explains how we create a new, extensive benchmark set based on publicly available data from the port of Barcelona, Spain. In Section 8 we describe the computational results that our approaches and the five formulations from the literature achieve on the new benchmark instances. Via a vast computational campaign, we detect instance characteristics that make some of the evaluated approaches competitive. Finally, in Section 9 we draw conclusions and highlight future research directions.

2 Formal definition and objective functions

We are given a set of n ships and a time horizon discretised into T intervals. Each ship i has an associated length $\ell_i > 0$, and arrival time $a_i \in \{1, \dots T\}$. The quay of length L is divided into m adjacent berths. Each berth j has an associated length $\lambda_j > 0$ and starts at a distance of $\delta_j \geq 0$ from the leftmost point of the quay ($\delta_1 = 0$, $\delta_j = \sum_{k=1}^{j-1} \lambda_k$). A ship i mooring with its leftmost end at berth j has an associated handling time $h_{ij} \in \mathbb{N}^+$. Such mooring is feasible at time t if $\delta_j + \ell_i \leq L$ (the ship fits in the quay), $t \geq a_i$ (the ship berths not earlier than its arrival), $t + h_{ij} \leq T$ (the cargo can be handled within the end of the time horizon), and the berths occupied by the ship are free of other ships during the entire handling period. We can represent the quay on the horizontal axis and the time on the vertical axis of a Cartesian plane, such as in Figure 3. Mooring ship i at berth j at time t corresponds to drawing a rectangle with the bottom left corner at coordinates (δ_j, t) and the top

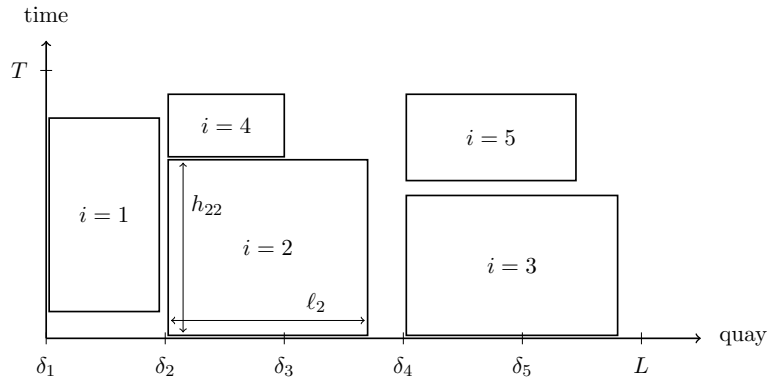


Figure 3: Geometric representation of five ships with feasible assignments.

right corner at coordinates $(\delta_j + \ell_i, t + h_{ij})$. The condition that no two ships occupy the same berth simultaneously implies that no two rectangles overlap.

2.1 Objective functions

Although the feasible set of many BAP versions is the same and represents allowed assignments of ships to berths and mooring times, different objective functions reflect the possible commercial and operational practices of ports around the world. In this following, we describe the objetive functions we test our proposed methodology on, which comprise the vast majority of the functions that have been used in the recent literature.

Bottleneck objective functions. The main bottleneck objective function is "max(compl)", that was used, e.g., by Lee et al. (2010b), Błażewicz et al. (2011), and Emde et al. (2014). Let V^{BAP} represent the set of assignments, whose elements are triples (i, j, t) indicating that ship i moors at berth j at time t. Let $X \subset P(V^{\mathrm{BAP}})$, where $P(\cdot)$ denotes the power set, be the set of feasible BAP solution. Objective "max(compl)" requires to minimise the completion time of the last ship served:

$$\min_{\boldsymbol{x} \in X} \left\{ \max_{(i,j,t) \in \boldsymbol{x}} (t + h_{ij}) \right\}.$$

The other bottleneck objective function, "max(pos)", was introduced by Lim (1988) but is not popular in recent literature. We describe it here for completeness because our method would support it, although we do not directly test it:

$$\min_{\boldsymbol{x} \in X} \left\{ \max_{(i,j,t) \in \boldsymbol{x}} (\delta_j + \ell_i) \right\}.$$

It requires to minimise the rightmost berth used by any serviced ship.

Cumulative objective functions. In this work, we focus on three popular cumulative objective functions, although our method would support any objective function that represents the total cost of a solution $\boldsymbol{x}$ as the sum of the costs of the assignments $(i, j, t) \in \boldsymbol{x}$. The three objective functions we consider are:

$$\sum(\text{compl}) \qquad \min_{\boldsymbol{x} \in X} \sum_{(i,j,t) \in \boldsymbol{x}} (t + h_{ij}) \tag{1a}$$

$$\sum w(\text{wait} + \text{hand}) \qquad \min_{\boldsymbol{x} \in X} \sum_{(i,j,t) \in \boldsymbol{x}} w_i \big[(t - a_i) + h_{ij} \big] \tag{1b}$$

$$\sum w(\text{wait} + \text{tard}) \qquad \min_{\boldsymbol{x} \in X} \sum_{(i,j,t) \in \boldsymbol{x}} w_i \big[(t - a_i) + (t + h_{ij} - s_i)_+ \big]. \tag{1c}$$

where $w_i \geq 0$ is a ship-specific weight that determines the importance of each ship, and s_i is the desired completion time of ship i.

3 Previous works

Most problems in the BAP family are $\mathcal{NP}$-hard (Hansen et al. 2003), and they have proven challenging to solve exactly. For example, the tree search method of Guan et al. (2004) takes over 15 hours (on average) for instances with 10 berths and 15 ships when minimising the weighted flow time. The compact formulation of Hansen et al. (2003) takes over 7 hours (on average) for instances with five berths and 30 ships, minimising the sum of waiting and handling times. The Lagrangian algorithm introduced by Imai et al. (2001) is faster (in the order of tens or hundreds of seconds) but has average gaps of 51% already for instances with five berths and 25 ships, again minimising the sum of waiting and handling times.

In our literature analysis, we found six formulations for problems in the BAP family which are competitive for at least some classes of instances. Out of them, five can be adapted to the problem types we consider. In particular, these five formulation directly support or can be straightforwardly adapted for the hybrid spatial attrbute that allows ships to occupy several berths simultaneously. The sixth model is the arc-flow model of Kramer et al. (2019). Unlike the other formulations, it strictly relies on the discrete spatial attribute, i.e., on the fact that each ship occupies exactly one berth. In the following, we provide a short résumé of these six formulations.

Guan et al. (2004) introduced two formulations for a BAP of type "hyb|dyn|fix|$\sum$w(wait+hand)" in which all berths have the same length. The first formulation, called Relative Position (RP), uses binary variables to keep track of pairs of ships such that the first completes before the second moors (along the temporal dimension), and such that the rightmost end of the first ship is on the left of the leftmost end of the second (along the spatial dimension). The second formulation, called Position Assignment (PA), is a hybrid packing-covering formulation with binary variables keeping track of the bottom-left corner of each ship's rectangle. We describe these formulations, respectively, in Appendix A.1 and Appendix A.2. In their work, the authors use RP as the main formulation and explore its solution space via tree search. A Lagrangian relaxation of PA is used during the tree's exploration to provide bounds and prune parts of the tree. The authors also include improvement heuristics to enhance the tree search performance. They test their approach on randomly generated instances with 10 berths, 168 intervals (one week, discretised hour-by-hour), and up to 80 ships.

Buhrkal et al. (2011) describes a formulation first introduced by Christensen et al. (2008) in their M.Sc. thesis. This is a generalised set partitioning problem (GSPP) formulation with one variable for each possible ship, berth, and feasible mooring time of the ship at the berth. We present it in Appendix A.3. Despite the large number of columns, the GSPP formulation enjoys an especially tight continuous relaxation that makes it extremely competitive, outperforming the previous state of the art (Monaco et al. 2007). However, it has two downsides. First, most practically relevant side-constraints involving more than one ship are hard (or outright impossible) to include in a GSPP-based formulation. By contrast, constraints involving single vessels can be incorporated straightforwardly, as shown, e.g., by Lalla-Ruiz et al. (2016). Second, despite the number of variables being polynomial in the input data (number of ships, number of berths, time horizon length), it tends to be very large. Indeed, in a subsequent study (Kramer et al. 2019), it was shown that building the GSPP model resulted in out-of-memory errors for instances with 200–250 ships on a machine with 16GB of RAM.

Ernst et al. (2017) originally considered a "cont|dyn|fix|$\sum$(compl)" BAP characterised by tidal constraints, i.e., a set of time windows during which the ships cannot leave the port. In Appendix A.4 and Appendix A.5, we describe variants of the two formulations presented by the authors, adapting them to the case of a discrete berth. The first is the sequence variables formulation (S), which uses two main sets of binary variables defined for each pair of ships. One set of variables identifies pairs of ships such that the first one completes before the second one starts processing, while the other set keeps track of ships with overlapping handling periods. The second is the time index formulation (TI), which takes this name because it includes one set of binary variables indexed over the ships and the time intervals, taking the value one if and only if the given ship completes handling by the given interval. Both formulations are strengthened with a large number of valid inequalities. The TI formulation outperforms S on randomly-generated instances incorporating tide times from the port of Newcastle,

Australia. These instances include up to 32 ships and 168 or 336 time intervals (one or two weeks, discretised hour-by-hour).

Kramer et al. (2019) introduced an arc-flow formulation for a "disc|dyn|pos|$\sum$w(wait+hand)" BAP. In their approach, each berth is a resource flowing through time; its path defines the ships served by the given berth. Because of this interpretation, the formulation heavily relies on the "disc" spatial attribute, i.e., that each ship occupies exactly one berth. On instances of this type, the arc-flow formulation is very competitive, even outperforming the previous state-of-the-art GSPP formulation.

Finally, as a consequence of the difficulty of solving the BAP exactly, we remark that most recent research has focused on heuristic methods, such as the first-fit-decreasing heuristic of Li et al. (1998), the genetic algorithm of Lee et al. (2010a) and the rounding heuristic of Błażewicz et al. (2011) for BAPs with handling times based on quay-crane assignment, the evolutionary algorithm of Cheong et al. (2010), the hybrid heuristic of Lalla-Ruiz et al. (2012), the iterated local search of Correcher et al. (2019), the dynamic-programming-based matheuristic of Nishi et al. (2020), and the algorithm portfolio approach of Wawrzyniak et al. (2020).

4 Solution approach for bottleneck objective functions

In this section, we introduce a methodology to solve the BAP with bottleneck objective functions, which we denote BAP_b. Because it is the most popular bottleneck objective in the literature, we use the "max(compl)" objective function throughout the section. Following existing literature, we show that the decision version of the BAP_b can be transformed into a special case of the Selective Graph Colouring Problem (SGCP). Furthermore, in this special case, the SGCP reduces the Maximum Stable Set Problem (MSSP). Using this reduction, we devise a bound-based search algorithm that progressively tightens upper and lower bounds on the optimum of the BAP_b.

4.1 The decision version of the BAP_b

The BAP_b with bottleneck objective function "max(compl)" asks to find a feasible assignment of ships to berths (called a *berthing plan*) that minimises the makespan. Given a deadline $M \in \{1, \dots, T\}$, the decision version of BAP_b (DBAP_b) asks if there is a berthing plan such that the corresponding makespan does not exceed M. We denote with $\text{DBAP}_\text{b}(M) \in \{\text{True}, \text{False}\}$ the solution of the DBAP_b with deadline M.

4.2 Solving DBAP_b using selective graph colouring

The SGCP, also known as the *Partition Colouring Problem* (Hoshino et al. 2011; Furini et al. 2018; Cornaz et al. 2019; Zhu et al. 2022), is a combinatorial optimisation problem defined on an undirected graph $G = (V, E)$ with vertices V and edges $E \subseteq V^2$, where the vertex set has an associated partition $\mathcal{P}$, i.e., $\bigcup \mathcal{P} = V$ and $P \cap Q = \emptyset \; \forall P, Q \in \mathcal{P}, P \neq Q$. The SGCP consists of selecting exactly one vertex in each partition set and assigning it a colour. The objective of the problem is to minimise the number of colours used, subject to the requirement that adjacent vertices cannot have the same colour. As a generalisation of the classical Vertex Colouring Problem, the SGCP is $\mathcal{NP}$-hard.

Figure 4 shows a small SGCP instance with nine vertices (represented by circles) partitioned into four sets (dashed rectangles). Solid grey lines denote edges. The figure shows an optimal solution using two colours (red and blue).

In the following theorem, we show how, for a given DBAP_b instance, we create a corresponding SGCP instance with the property that the DBAP_b solution is positive if and only if the optimal solution of the SGCP uses one colour. Then, in Section 4.3, we show that the SGCP uses one colour if and only if the corresponding MSSP on the same graph has an optimal solution of size n, i.e., the number of ships.

Theorem 1. *Given an instance of DBAP$_\text{b}$ with threshold M, it is possible to construct an undirected graph $G_M^{\text{BAP}} = (V_M^{\text{BAP}}, E_M^{\text{BAP}})$ such that* $\text{DBAP}_\text{b}(M) = \text{True}$ *if and only if the Selective Graph Colouring*

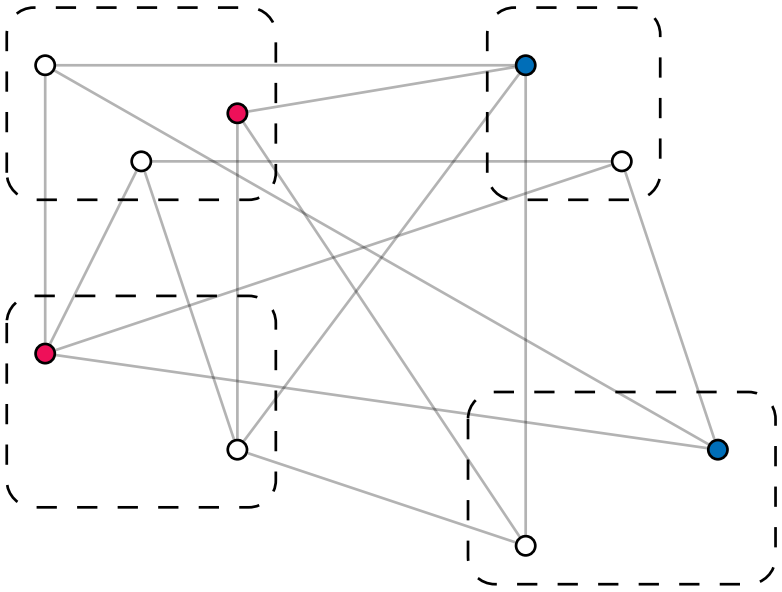


Figure 4: Example SGCP instance. Dashed rectangles represent partitions, and solid grey lines are edges. A possible optimal solution using red and blue colours is depicted.

Problem on G_M^{BAP} has optimal objective value equal to one.

Proof. Recall that a DBAP_b instance contains n ships (each of length ℓ_i), m berths (each starting at distance δ_j from the leftmost point of the quay, which has total length L), that each ship is available at time a_i and that the handling time of a ship i berthing at j is h_{ij}. We first build the vertex set of the graph:

$$V_M^{\text{BAP}} = \{(i,j,t) \ : \ 1 \leq i \leq n,\ 1 \leq j \leq m,\ a_i \leq t \leq M - h_{ij}, \text{ and } \delta_j + \ell_i \leq L\}.$$

Vertex (i,j,t) corresponds to ship i mooring at berth j at time t. We partition V_M^{BAP} by grouping vertices corresponding to the same ship, i.e., $\mathcal{P}_M = \{V_1^M, \ldots, V_n^M\}$ where $V_i^M = \{(i',j',t') \in V_M^{\text{BAP}} \ : \ i' = i\}$. To build the edge set E_M^{BAP}, we add an edge between vertices (i,j,t) and (i',j',t'), for $i \neq i'$, if the area of overlap of the corresponding rectangles $[t, t+h_{ij}] \times [\delta_j, \delta_j + \ell_i]$ and $[t', t'+h_{i'j'}] \times [\delta_{j'}, \delta_{j'} + \ell_{i'}]$ is strictly positive. Such an edge stands for an incompatible assignment that would cause the same berth to be occupied by two ships simultaneously. Because each ship must be assigned a single mooring berth (i.e., the berth where the leftmost end of the ship starts) and a single mooring time, we must select exactly one vertex in each partition set. If the optimal solution of the SGCP uses one colour, it is possible to select one vertex in each partition so that no two vertices are linked by an edge. Each vertex corresponds to a mooring berth and a mooring time for a ship; the lack of edges among the vertices identifies a feasible solution with no overlap. By construction of V_M^{BAP}, these vertices ensure that all ships can be accommodated within the deadline M. In this case, therefore, the solution of DBAP_b is True. Conversely, if the optimal solution of the SGCP uses two or more colours, the solution of DBAP_b is False. □

Demange et al. (2015) first highlighted the relationship between the BAP and the SGCP in a survey on SGCP applications. The authors remarked that "It would be interesting to explore graph theoretical approaches [...] to solve the berth allocation problem". Preliminary work in this direction was conducted in the PhD thesis of Santini (2017). The authors also remark that the SGCP remains $\mathcal{NP}$-hard when restricted to graphs G_M^{BAP} because these graphs generalise interval graphs, where the SGCP is $\mathcal{NP}$-complete. We highlight that the correspondence between DBAP_b and the SGCP allows modelling additional berthing constraints. For example, we can model ship–berth draft compatibilities (Grubišić et al. 2015; Liu et al. 2018; Malaguti et al. 2018) by removing vertices corresponding to incompatible ship–berth pairs.

$\text{DBAP}_\text{b}(\underline{M})$	$\text{DBAP}_\text{b}(\bar{M})$	New $[\underline{z}, \bar{z}]$	New $[\underline{z}, \bar{z}]$ with tightened bounds
True	True	$[\underline{z}, \underline{M}]$	$[\underline{z}, z_{\text{DBAP}_\text{b}(\underline{M})}]$
False	True	$[\underline{M}, \bar{M}]$	$[\underline{M}, z_{\text{DBAP}_\text{b}(\bar{M})}]$
False	False	$[\bar{M}, \bar{z}]$	$[\bar{M}, \bar{z}]$

Table 1: Update rule for bounds $\underline{z}$ and $\bar{z}$ in the bound-based search algorithm.

4.3 Solving DBAP_b using maximum stable sets

Remark that the optimal solution of a SGCP instance does not change if each partition set is transformed into a clique. Therefore, we add to the edge set E_M^{BAP} all the following edges:

$$\hat{E}_M^{\text{BAP}} = E_M^{\text{BAP}} \cup \big\{\{(i,j,t),(i',j',t')\} \in V_M^{\text{BAP}} \times V_M^{\text{BAP}} \ : \ i = i',\ (j,t) \neq (j',t')\big\}.$$

The (unweighted) MSSP asks to find a stable set of maximal cardinality in a given graph. The following theorem establishes an alternative condition on when $\text{DBAP}_\text{b}(M) = \text{True}$ or False.

Theorem 2. *Given an instance of DBAP$_b$ with threshold M, $\text{DBAP}_\text{b}(M) = \text{True}$ if and only if the Maximum Stable Set Problem on $\hat{G}_M^{\text{BAP}} = (V_M^{\text{BAP}}, \hat{E}_M^{\text{BAP}})$ has optimal objective value equal to n, the number of ships in the DBAP$_b$ instance.*

Proof. Because in $\hat{G}_M^{\text{BAP}}$ all partitions of G_M^{BAP} are turned into cliques, any stable set can contain at most one vertex from each partition, i.e., at most one vertex associated with each ship. Any such stable set of size n, therefore, contains exactly one vertex in each of the n partitions, i.e., assignes a feasible berth and time to each ship. Colouring the vertices of the optimal MSSP stable set with the same colour gives a feasible solution for the SGCP on G_M^{BAP} that uses one colour, and thus, by Theorem 1, $\text{DBAP}_\text{b}(M) = \text{True}$. Conversely, if the largest stable set of $\hat{G}_M^{\text{BAP}}$ has size strictly smaller then n, it is not possible to cover all partitions of G_M^{BAP} with a single stable set and, therefore, the optimal solution of the SGCP uses at least two colours. By Theorem 1, then, $\text{DBAP}_\text{b}(M) = \text{False}$. □

4.4 Bound-based search algorithm

This section presents an algorithm to find the BAP_b optimal solution $z^* \in \{H, \dots, T\}$, where

$$H = \max_{i \in \{1,\dots,n\}} \ \min_{j \in \{1,\dots,m\}} (a_i + h_{ij})$$

is a lower bound on the optimal solution of the BAP_b. We first present a basic version in Section 4.4.1 and then improve it via dynamic time discretisation in Section 4.4.2. In Section 6.1, we give more implementation details about this procedure and present further improvements.

4.4.1 Base algorithm

The algorithm we introduce keeps track of two bounds, $\underline{z}$ and $\bar{z}$ such that $\underline{z} < z^* \leq \bar{z}$. When $\underline{z} = \bar{z} - 1$, the optimum of the BAP_b is $z^* = \bar{z}$ and the algorithm terminates.

Initially, we set $\underline{z} = H - 1$ and $\bar{z} = T$. At each iteration, we consider the two values $\underline{M}$ and $\bar{M}$ at one-third and two-thirds of the interval $[\underline{z}, \bar{z}]$. More precisely, we let

$$\underline{M} = \underline{z} + \left\lfloor \frac{1}{3}(\bar{z} - \underline{z}) \right\rfloor \quad \text{and} \quad \bar{M} = \underline{z} + \left\lceil \frac{2}{3}(\bar{z} - \underline{z}) \right\rceil. \tag{2}$$

We solve $\text{DBAP}_\text{b}(\underline{M})$ and $\text{DBAP}_\text{b}(\bar{M})$ using the MSSP, as explained in Section 4.3. We then update the values of $\underline{z}$ and $\bar{z}$ as outlined in the first three columns of Table 1. The algorithm terminates when the criterion $\underline{z} = \bar{z} - 1$ is met. The advantage of this search-by-thirds approach over classical binary search is that the two evaluations of $\text{DBAP}_\text{b}(\underline{M})$ and $\text{DBAP}_\text{b}(\bar{M})$ can be executed in parallel.

Recovering the BAP_b solution from the MSSP solution. Assume that $\text{DBAP}_\text{b}(M) = \text{True}$ for some value of M. In this case, the corresponding MSSP instance has a solution of size n. Let S be the independent set making up the MSSP solution. The makespan of this solution is

$$z_{\text{DBAP}_\text{b}(M)} = \max_{(i,j,t)\in S} \{t + h_{ij}\}.$$

Bound tightening. When $\text{DBAP}_\text{b}(M) = \text{True}$, there is a solution to the BAP_b whose makespan is at most M. However, the makespan $z_{\text{DBAP}_\text{b}(M)}$ of the corresponding berthing plan can be strictly smaller than M when the BAP_b solution does not use the entire planning horizon. In this case, using the BAP_b makespan instead of M in the bound-based search algorithm is convenient because the makespan provides tighter bounds when updating the interval $[\underline{z}, \bar{z}]$. The fourth column of Table 1 provides the details of this update.

4.4.2 Dynamic time discretisation

A key difficulty in solving DBAP_b by transformation to the MSSP is that the resulting vertex set V_M^{BAP} is large; indeed, $|V_M^{\text{BAP}}| = \mathcal{O}(nmT)$. In practical applications, term T is larger than n and m. Therefore, in this section, we introduce a time discretisation technique to reduce the number of periods considered and, thus, the size of V_M^{BAP}.

Given a coefficient $\alpha \in \mathbb{N}^+$, we introduce two methods to transform a DBAP_b instance into a new one with a more coarse time discretisation, containing roughly T/α periods. We denote the two resulting instances $\text{DBAP}_\text{b}^{1,\alpha}$ and $\text{DBAP}_\text{b}^{2,\alpha}$. Their solutions, given a deadline M, are denoted $\text{DBAP}_\text{b}^{1,\alpha}(M)$ and $\text{DBAP}_\text{b}^{2,\alpha}(M)$.

Properties. The two discretisations satisfy the following properties for any $\alpha \geq 2$. If $\text{DBAP}_\text{b}^{1,\alpha}(M) = \text{True}$, then $\text{DBAP}_\text{b}(M) = \text{True}$; however, if $\text{DBAP}_\text{b}^{1,\alpha}(M) = \text{False}$, $\text{DBAP}_\text{b}(M)$ could be either True or False. Conversely, if $\text{DBAP}_\text{b}^{2,\alpha}(M) = \text{False}$, then $\text{DBAP}_\text{b}(M) = \text{False}$; however, if $\text{DBAP}_\text{b}^{2,\alpha}(M) = \text{True}$, $\text{DBAP}_\text{b}(M)$ could be either True or False. Intuitively, in $\text{DBAP}_\text{b}^{1,\alpha}$, the problem data is modified conservatively (i.e., pessimistically) producing a more constrained instance. Therefore, if $\text{DBAP}_\text{b}^{1,\alpha}$ is feasible for a given M, the original DBAP_b instance is also feasible. Conversely, in $\text{DBAP}_\text{b}^{2,\alpha}$, the problem data is modified optimistically producing a relaxed instance. Therefore, if $\text{DBAP}_\text{b}^{2,\alpha}$ is infeasible for a given M, the original DBAP_b instance is also infeasible.

Formal definition. Instance $\text{DBAP}_\text{b}^{1,\alpha}$ is obtained from the following modifications of the original DBAP_b instance: (i) the number of periods T is replaced by $\lfloor T/\alpha \rfloor$; (ii) each ship arrival time a_i is replaced by $\lceil a_i/\alpha \rceil$; (iii) each ship handling time h_{ij} is replaced by $\lceil h_{ij}/\alpha \rceil$; (iv) the deadline M is replaced by $\lfloor M/\alpha \rfloor$. Conversely, in instance $\text{DBAP}_\text{b}^{2,\alpha}$, the number of periods is replaced by $\lceil T/\alpha \rceil$, the arrival times are $\lfloor a_i/\alpha \rfloor$, the handling times are $\lfloor h_{ij}/\alpha \rfloor$, and the deadline is $\lceil M/\alpha \rceil$. We remark that, for $\alpha = 1$, the above definition guarantees that both $\text{DBAP}_\text{b}^{1,\alpha}$ and $\text{DBAP}_\text{b}^{2,\alpha}$ correspond to the original instance.

Primal bound from the discretised instance. When $\text{DBAP}_\text{b}^{1,\alpha}(\bar{M}) = \text{True}$ for a deadline $\bar{M}$, we know that $\text{DBAP}_\text{b}(\bar{M}) = \text{True}$. Therefore, $\bar{M}$ is an upper bound on the optimum z^*. However, in Section 4.4.1, we have shown that this bound can be improved to $z_{\text{DBAP}_\text{b}(M)}$ when solving $\text{DBAP}_\text{b}(M)$. We can also improve the bound when solving $\text{DBAP}_\text{b}^{1,\alpha}(\bar{M})$. To this end, let S^1 be the only independent set of the graph associated with the solution of $\text{DBAP}_\text{b}^{1,\alpha}(\bar{M})$. We recall that $\text{DBAP}_\text{b}^{1,\alpha}$ has a coarser time discretisation than DBAP_b and a period t in solution P^1 corresponds to a period αt in the original discretisation of DBAP_b. Therefore, the following scaled makespan is a valid upper bound for z^*:

$$z_{\text{DBAP}_\text{b}^{1,\alpha}(M)} = \max_{(i,j,t)\in S^1} \{\alpha t + h_{ij}\}.$$

Usage. When using dynamic time discretisation, we compute $\text{DBAP}_\text{b}^{2,\alpha_2}(\underline{M})$ instead of $\text{DBAP}_\text{b}(\underline{M})$ and $\text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M})$ instead of $\text{DBAP}_\text{b}(\bar{M})$, where α_1 and α_2 are two integer numbers. In our implementation, both parameters initially start at the same value, i.e., $\alpha_1 = \alpha_2 = \alpha^{(0)}$. Our algorithm uses a sequence of Q integer numbers $\alpha^{(0)}, \alpha^{(1)}, \ldots, \alpha^{(Q)}$ with $\alpha^{(q)} > \alpha^{(q+1)}$ for all $q \in \{0, \ldots, Q-1\}$ and $\alpha^{(Q)} = 1$. This sequence constitutes the update schedule of the discretisation parameter; if the algorithm does not terminate early, eventually the last element of the schedule is $\alpha^{(Q)} = 1$ and one resorts to solving the original DBAP_b problem without a coarser time discretisation. At each iteration of the algorithm, the following three cases can happen:

1. If $\text{DBAP}_\text{b}^{2,\alpha_2}(\underline{M}) = \text{False}$ and $\text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M}) = \text{True}$, we know that $\text{DBAP}_\text{b}(\underline{M}) = \text{False}$ and $\text{DBAP}_\text{b}(\bar{M}) = \text{True}$. Therefore, we can update the interval $[\underline{z}, \bar{z}]$ as $[\underline{M}, z_{\text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M})}]$ and proceed to the next iteration.

2. If $\text{DBAP}_\text{b}^{2,\alpha_2}(\underline{M}) = \text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M}) = \text{False}$, we know that $\text{DBAP}_\text{b}(\underline{M}) = \text{False}$, but $\text{DBAP}_\text{b}(\bar{M})$ can be True or False. To proceed, we must determine which of the two cases occurs. To this end, we increase the precision of the time discretisation. Let the current discretisation parameter be $\alpha_1 = \alpha^{(q)}$ for some $q \in \{0, \ldots, Q-1\}$. Then, we update it as $\alpha_1 \leftarrow \alpha^{(q+1)}$ and solve $\text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M})$ again. We keep decreasing the value of α_1 following the schedule until either $\text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M}) = \text{True}$ or $\alpha_1 = \alpha^{(Q)} = 1$ and $\text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M}) = \text{False}$. In the first case, we update the interval $[\underline{z}, \bar{z}]$ as $[\underline{M}, z_{\text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M})}]$. In the second case, we update it as $[\bar{M}, \bar{z}]$.

3. If $\text{DBAP}_\text{b}^{2,\alpha_2}(\underline{M}) = \text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M}) = \text{False}$, we know that $\text{DBAP}_\text{b}(\bar{M}) = \text{True}$, but $\text{DBAP}_\text{b}(\underline{M})$ can be True or False. Again, to proceed, we must determine which of the two cases occurs. Let the current discretisation parameter be $\alpha_2 = \alpha^{(q)}$ for some $q \in \{0, \ldots, Q-1\}$. Then, we update $\alpha_2 \leftarrow \alpha^{(q+1)}$ and solve $\text{DBAP}_\text{b}^{2,\alpha_2}(\underline{M})$ again. Eventually, we either reach a value of α_2 for which $\text{DBAP}_\text{b}^{2,\alpha_2}(\underline{M}) = \text{False}$, or $\alpha_2 = \alpha^{(Q)} = 1$ and $\text{DBAP}_\text{b}^{2,\alpha_2}(\underline{M}) = \text{True}$. In the first case, we update the interval $[\underline{z}, \bar{z}]$ as $[\underline{M}, z_{\text{DBAP}_\text{b}^{1,\alpha_1}(\bar{M})}]$. In the second case, we update it as $[z, z_{\text{DBAP}_\text{b}(\underline{M})}]$. We remark that we can compute $z_{\text{DBAP}_\text{b}(\underline{M})}$ because, when $\alpha_2 = 1$, solving $\text{DBAP}_\text{b}^{2,\alpha_2}(\underline{M})$ corresponds to solving the original $\text{DBAP}_\text{b}(\underline{M})$.

5 Solution approach for cumulative objective functions

Let $\hat{G}^{\text{BAP}} = (V^{\text{BAP}}, \hat{G}^{\text{BAP}}) = (V_T^{\text{BAP}}, \hat{G}_T^{\text{BAP}})$ be the conflict graph relative to the entire time horizon up to period T. In the BAP with cumulative objective functions (which we denote BAP_c), we minimise the sum over all ships of a cost that depends on the ship, the berth, and the mooring time. For a given assignment $v \in V^{\text{BAP}}$, we denote this cost with $c_v \geq 0$. Different ways to assign values to c_v produce different cumulative objective functions. For example, for the "Σ(compl)" objective of (1a), $c_v = t(v) + h_{i(v),j(v)}$, where $i(v) = i$, $j(v) = j$, and $t(v) = t$ for an assignment $v = (i, j, t)$. The following theorem shows that any BAP_c instance can be solved via the solution of a single Maximum Weighted Stable Set Problem (MWSSP) instance. The MWSSP generalises the MSSP: each vertex is associated with a positive weight, and the objective consists in finding the stable set with the largest total weight. The MWSSP reduces to the MSSP when all weights are 1.

Theorem 3. *Given a BAP_c instance, it is possible to assign weights to the vertices of graph $\hat{G}^{\text{BAP}}$ in such a way that optimal MWSSP solutions correspond to optimal BAP_c solutions. In other words, a stable set of $\hat{G}^{\text{BAP}}$ is optimal for the MWSSP if and only if the corresponding berthing plan is optimal for the BAP_c.*

Before proving Theorem 3, we prove two preliminary lemmas. The first allows to sparsify V^{BAP}, while the second establishes a necessary condition for MWSSP optimality on $\hat{G}^{\text{BAP}}$.

Lemma 1. *Let $\{V_1, \ldots, V_n\}$ be the partition of V^{BAP} in which vertices relative to ship i belong to the same set V_i. Let m_i be the minimum cost associated with a ship $i \in \{1, \ldots, n\}$, i.e.,*

$$m_i = \min_{v \in V_i} c_v.$$

Let $d_v = c_v - m_{i(v)}$ be the cost differential of an assignment v compared to the minimum possible cost of an assignment for the same ship. Denote with $M_0 = \sum_{i=1}^{n} m_i$ a lower bound on the total cost of a BAP_c solution and with U_S the cost of a feasible solution S, i.e., $U_S = \sum_{v \in S} c_v$. Given any feasible solution S, no vertex $v \in V^{\mathrm{BAP}}$ such that $d_v > U_S - M_0$ can be part of an optimal solution.

Proof. Denote with U'_S the quantity $U_S - M_0$ and let $\hat{S}$ be any solution that uses v. We want to show that $U_{\hat{S}} > U_S$. Consider, then, $U'_{\hat{S}}$:

$$\begin{aligned} U'_{\hat{S}} = U_{\hat{S}} - M_0 &= \sum_{u \in \hat{S}} c_u - \sum_{i=1}^{n} m_i \\ &= \sum_{u \in \hat{S}} (d_u + m_{i(u)}) - \sum_{i=1}^{n} m_i \\ &= \sum_{u \in \hat{S}} d_u + \sum_{i=1}^{n} m_i - \sum_{i=1}^{n} m_i \\ &= \sum_{u \in \hat{S}} d_u = d_v + \sum_{\substack{u \in \hat{S} \\ u \neq v}} d_u \\ &\geq d_v > U_S - M_0. \end{aligned}$$

Thus, $U'_{\hat{S}} + M_0 > U_S$, i.e., $U_{\hat{S}} > U_S$. □

We now assign weights to the vertices of $\hat{G}^{\mathrm{BAP}}$. Given a feasible solution S, let $K_S = U'_S + 1 = U_S - M_0 + 1$. Then, we assign to each vertex $v \in V^{\mathrm{BAP}}$ the following non-negative weight:

$$p_v = K_S - d_v.$$

Lemma 2. *Any optimal solution of the MWSSP on $\hat{G}^{\mathrm{BAP}}$ with weights p_v contains n vertices.*

Proof. Because each of the n sets $V_1, \ldots, V_n$ has been turned into a clique in $\hat{E}^{\mathrm{BAP}}$, no solution of the MWSSP (i.e., no stable set of $\hat{G}^{\mathrm{BAP}}$) can have more than n vertices. Thus, we must show that no solution with less than n vertices can be optimal.

Lemma 1 allows us to remove any vertex $v \in V^{\mathrm{BAP}}$ with $d_v > U'_S > K_S$. Therefore, for the remaining vertices, $d_v < K_S$, and thus $p_v > 0$, ensuring that no vertex has a negative weight.

Moreover, because $d_v \geq 0$, $p_v \leq K_S$ for all vertices $v \in V^{\mathrm{BAP}}$. Therefore, no stable set with $(n-1)$ vertices can have weight strictly larger than $(n-1)K_S$. Then, we prove that there is at least one solution with n vertices that has total weight strictly larger than $(n-1)K_S$. Indeed, we show that the feasible solution S used to compute the upper bound U_S and the parameter K_S satisfies this property.

The total weight of solution S is:

$$\begin{aligned} \sum_{v \in S} p_v &= \sum_{v \in S} (K_S - d_v) = nK_S - \sum_{v \in S} d_v \\ &= nK_S - \sum_{v \in S} c_v + \sum_{v \in S} m_{i(v)} \\ &= nK_S - \sum_{v \in S} c_v + \sum_{i=1}^{n} m_i \\ &= nK_S - U_S + M_0 = nK_S - U'_S = nK_S - (K_S - 1) \\ &= (n-1)K_S + 1 > (n-1)K_S. \end{aligned}$$

□

Finally, we prove Theorem 3.

Proof of Theorem 3. Let $\mathcal{S}$ be the collection of stable sets of $\hat{G}^{\text{BAP}}$ and let $\hat{S} \in \mathcal{S}$ be an optimal solution of the MWSSP. Due to Lemma 2, this solution contains n vertices. Its total weight, which is maximal among the stable sets of $\hat{G}^{\text{BAP}}$ is:

$$\sum_{v \in \hat{S}} p_v = \sum_{v \in \hat{S}} (K_S - d_v) = nK_S - \sum_{v \in \hat{S}} d_v. \tag{3}$$

Because nK_S is a constant and cost (3) is maximal, the term $\sum_{v \in \hat{S}} d_v = U'_{\hat{S}}$ is minimal. Adding constant M_0, $\hat{S}$ is a minimiser of the following objective function:

$$\hat{S} \in \arg\min_{S \in \mathcal{S}} (U'_S + M_0) \quad \Rightarrow \quad \hat{S} \in \arg\min_{S \in \mathcal{S}} U_S,$$

i.e., $\hat{S}$ corresponds to a BAP_c solution of minimal cost. □

The construction presented in this section relies on the availability of a feasible solution S. However, if such a solution were not available, we remark that the corresponding value K_S used to define weights p_v can be replaced by a sufficiently large number. For example, using constant $K = 1 + \sum_{i=1}^{n} \left(\max_{v \in V_i} d_i\right)$ instead of K_S ensures that all the results presented in this section still hold.

6 Implementation

In this section, we give additional details on the implementation of the solution methods introduced in Section 4 and Section 5 for the BAP_b and BAP_c, respectively.

6.1 Aspects related to the algorithm for BAP_b

Initial feasible solution. We obtain an initial feasible solution S using a constructive insertion heuristic. We first consider the ships in non-decreasing order of arrival time, breaking ties by non-increasing ship length. When considering ship i, we examine every berth j such that $\delta_j + \ell_i \leq L$ and determine the earliest time $t \geq a_i$ at which the corresponding rectangle does not overlap any previously scheduled ship and $t + h_{ij} \leq T$. Among the feasible assignments, we select one minimising $t + h_{ij}$, breaking ties by choosing the smallest t and then the smallest j. To reduce the dependence on the ship order while keeping the construction inexpensive, we repeat the procedure considering the ships in non-increasing order of length, breaking ties by non-decreasing arrival time, and retain the solution with the smallest makespan.

Initial solution improvement. We use the initial solution as a warmstart to two Constraint Programming (CP) models solved via CP-SAT (Perron et al. 2023) with a time limit of one minute. These models are described in detail in Appendix B and are denoted "Single-rectangle" and "Optional-rectangle". Preliminary experiments showed that the CP models are not competitive for the DBAP_b (see also, e.g., Zampelli et al. 2013), but are quite effective at improving the incumbent solution quickly. Furthermore, remark that the models are compact and do not require building the edge sets E_T^{BAP} or $\hat{E}_T^{\text{BAP}}$; hence, their memory footprints are small. Once the initial solution S is available, denote with z_S its makespan. Then, instead of initialising $\bar{z} = T$ in Section 4.4.1, we set $\bar{z} = z_S$.

Graph pruning. When solving $\text{DBAP}_\text{b}(M)$, vertices (i, j, t) such that $t + h_{ij} > M$ are removed by construction of V_M^{BAP}. Moreover, we remove vertices conflicting with all the vertices of a different ship. More precisely, if there is a vertex $(i_1, j, t) \in V_{i_1}^M$ that is adjacent to all vertices of $V_{i_2}^M$ for some $i_2 \neq i_1$, then (i_1, j, t) is removed. The procedure is repeated until no such vertex remain. Removing these vertices is much more likely to happen for times t near the deadline M, as assignments might block enough of the quay to prevent another ship from completing within the deadline.

Furthermore, denote with $N(i,j,t)$ the vertices adjacent to $(i,j,t) \in V_M^{\text{BAP}}$ in G_M^{BAP}. The elements of $N(i,j,t)$ are assignments relative to other ships that are incompatible with ship i mooring at berth j at time t. Given two assignments (i,j,t) and (i,j',t') relative to the same ship i and such that $t + h_{ij} \leq t' + h_{ij'}$, if $N(i,j,t) \subseteq N(i,j',t')$, then vertex (i,j,t') can be removed because it is dominated by (i,j,t). Indeed, (i,j,t) only causes a subset of the conflicts caused by (i,j',t'), and has a shorter completion time; therefore, (i,j,t) is always preferrable to (i,j',t').

After applying the above pruning rule, it is still possible to find assignments (i,j,t) and (i,j',t) related to the same ship i and mooring time t but different berths j and j', such that $t + h_{ij} = t + h_{ij'}$ and $N(i,j,t) = N(i,j',t)$. These two assignments imply the same completion time and cause the same conflicts. Therefore, selecting one over the other has no impact on the overall solution and introduces symmetries in the problem. To remove these symmetries, we arbitrarily remove the vertex corresponding to the rightmost berth, i.e., we remove (i,j,t) if $j > j'$ or (i,j',t) if $j' > j$.

Instance decomposition. If there are large time periods without ship arrivals, graph $\hat{G}^{\text{BAP}}$ might contain more than one connected component. In that case, the MSSP can be solved separately for each connected component, and the corresponding solutions can be merged into an overall solution.

Refining $\underline{M}$ and $\bar{M}$ To reduce the number of evaluations of $\text{DBAP}_\text{b}(\underline{M})$ and $\text{DBAP}_\text{b}(\bar{M})$, we make sure that both thresholds are achievable by a feasible solution. Let

$$\tilde{T} = \{t + h_{ij} \,|\, (i,j,t) \in V_M^{\text{BAP}}\}$$

be the set of feasible completion times for the ships. In general, $\tilde{T} \subseteq \{H, \ldots, T\}$, but the inclusion can be strict if not all times are valid completion times, depending on the values of a_i and h_{ij}. As an extreme example, if all values of a_i and h_{ij} are even numbers, $\tilde{T}$ does not contain any odd number. In our procedure, after updating the values of $\underline{M}$ and $\bar{M}$ as per (2), we can refine them as

$$\underline{M} \leftarrow \min\{t \in \tilde{T} \,|\, t \geq \underline{M}\} \quad \text{and} \quad \bar{M} \leftarrow \max\{t \in \tilde{T} \,|\, t \leq \bar{M}\}.$$

6.1.1 Solving the MSSP

We solve the MSSP with a portfolio of algorithms from the literature. In the first phase, we employ fast heuristic algorithms to find a stable set of cardinality n. As a first step, we run the greedy algorithms implemented in the StableSolver suite (Fontan 2020), which are based on the algorithms of Sakai et al. (2003). These are MWSSP algorithms, and therefore, we set all weights to 1 in the MSSP case. The four algorithms (gwmin, gwmax, gwmin2, and strong) run in parallel; if any of them finds a stable set of cardinality n, we stop and report a positive answer to $\text{DBAP}_\text{b}(M)$. Otherwise, as a second step, we run the two row-weighting local-search algorithms provided by the same suite, which adapt the methods of Lin et al. (2020) and Gao et al. (2015), respectively. Again, if any of the two algorithms finds a stable set of size n, $\text{DBAP}_\text{b}(M) = \text{True}$. Otherwise, we store the largest stable set found in the first phase. We remark that, by default, the local-search algorithms' code calls gwmin to generate a starting solution. In our implementation we modified the code of Fontan (2020) to allow both algorithms to start from a user-provided starting solution, i.e., the solution found in the first step.

In the second phase, we employ exact algorithms for the MSSP and the Maximum-Clique Problem (MCP). The MCP asks to find the largest clique in a graph; a clique is a subset of vertices that are all adjacent to each other. A maximum stable set in a graph corresponds to a maximum clique in the complementary graph, i.e., the graph that contains an edge when the original graph does not contain that edge, and vice versa. Therefore, MCP algorithms can also be used to solve the MSSP. If a graph is dense, then then its complementary graph is spares and vice-versa. Using solvers for both the MSSP and the MCP exploits this property without increasing the memory footprint too much, since the memory usage is dominated by the storage requirements of the densest of the two graphs.

We use three algorithms in parallel, and stop as soon as one of them finds a stable set of size n ($\text{DBAP}_\text{b}(M) = \text{True}$) or certifies that none exists ($\text{DBAP}_\text{b}(M) = \text{False}$). The three algorithms are:

- Tclique, a MCP solver part of the SCIP optimisation suite (Hojny et al. 2025). The advantage of this solver is that the graph can be supplied through adjacency callbacks, so its edges are never stored. The algorithm takes the current best solution's size as a lower bound to speed up the search, and allows the user to specify a callback on a new solution, which allows early termination in case a solution of size n is found.
- The weighted branch-and-reduce algorithm of KaMIS (Lamm et al. 2019).
- The algorithm of Held et al. (2012). This algorithm uses the best current solution's size as a lower bound to speed up the search, and a goal parameter (n in our case) representing the target size of the stable set we are searching.

All three algorithms work on the weighted versions of the respective (maximum clique and maximum stable set) problems. To use them in our unweighted case, we set all weights to 1. Figure 5 gives a schematic view of the overall algorithm used to solve DBAP_b. In our implementation, the algorithm uses up to four parallel threads, which is a conservative requirement for any modern CPU as of 2026.

6.2 Aspects related to the algorithm for BAP_c

Initial feasible solution. We solve the BAP_c via the described transformation to the MWSSP using an initial feasible solution S to provide the upper bound U_S. The quality of this solution affects computational performance: a lower U_S permits more vertex pruning through Lemma 1 and results in a tighter initial lower bound for the MWSSP algorithm. To obtain a feasible solution, we first apply the constructive insertion heuristic with each of the two ship orderings described for the BAP_b. In addition, we run a cost-oriented version of both ship orderings used in that procedure. When considering ship i, this version selects, among the assignments that do not overlap with the ships already in the partial solution, one with minimum cost c_v, breaking ties by smallest completion time and then by smallest berth index. We retain the solution of minimum total cost and denote it by S, with cost U_S.

Initial solution improvement. We attempt to improve the initial solution using the same CP approach described for BAP_b, after appropriately changing the objective function. Appendix B details the base CP models and the modifications required by the cumulative objectives. After running the CP solvers with a short time limit of one minute, if none of the runs proved optimality, we further improve the solutions with the following reinsertion procedure, applied to each solution. We consider the ships in non-increasing order of the costs of their current assignments. For each ship, we temporarily remove its assignment and examine all vertices in the corresponding set V_i that are compatible with the assignments of the other ships. We replace the removed assignment with a minimum-cost compatible vertex whenever this strictly decreases U_S. After considering all ships, we repeat the procedure until a complete pass produces no improvement. We then supply the improved solution as a hint to a further 1-minute run of the CP solver. Again, if the CP models prove optimality during this run, the BAP_c instance is solved and no MWSSP is constructed. Otherwise, the best initial solution replaces S before U_S, K_S, and the weights p_v are computed.

Graph pruning. As mentioned above, Lemma 1 gives a BAP_c-specific pruning technique. Moreover, all the graph pruning techniques presented in Section 6.1 can be adapted for the BAP_c by considering assignment costs c_v instead of completion times.

The BAP_c graph can be further pruned using the following technique, based on a Lagrangian lower bound on the cost of the optimal solution. In this bound, the quay capacity is relaxed and its violations are penalised in a Lagrangian fashion. Let us first consider the trivial lower bound $\sum_{i\in\{1,\dots,n\}} m_i$, where m_i is the lowest cost of an assignment associated to ship i, and was defined in Lemma 1. A solution using these assignments might be unfeasible because, at any given time, there might be overlaps among berthed ships, and these overlaps might not be resolved by moving ships to other parts of the quay, simply because there is not enough berth length to accommodate all ships. The proposed bound uses Lagrangian multipliers to price quay capacity and, in this way, obtains a much tighter bound than the trivial one.

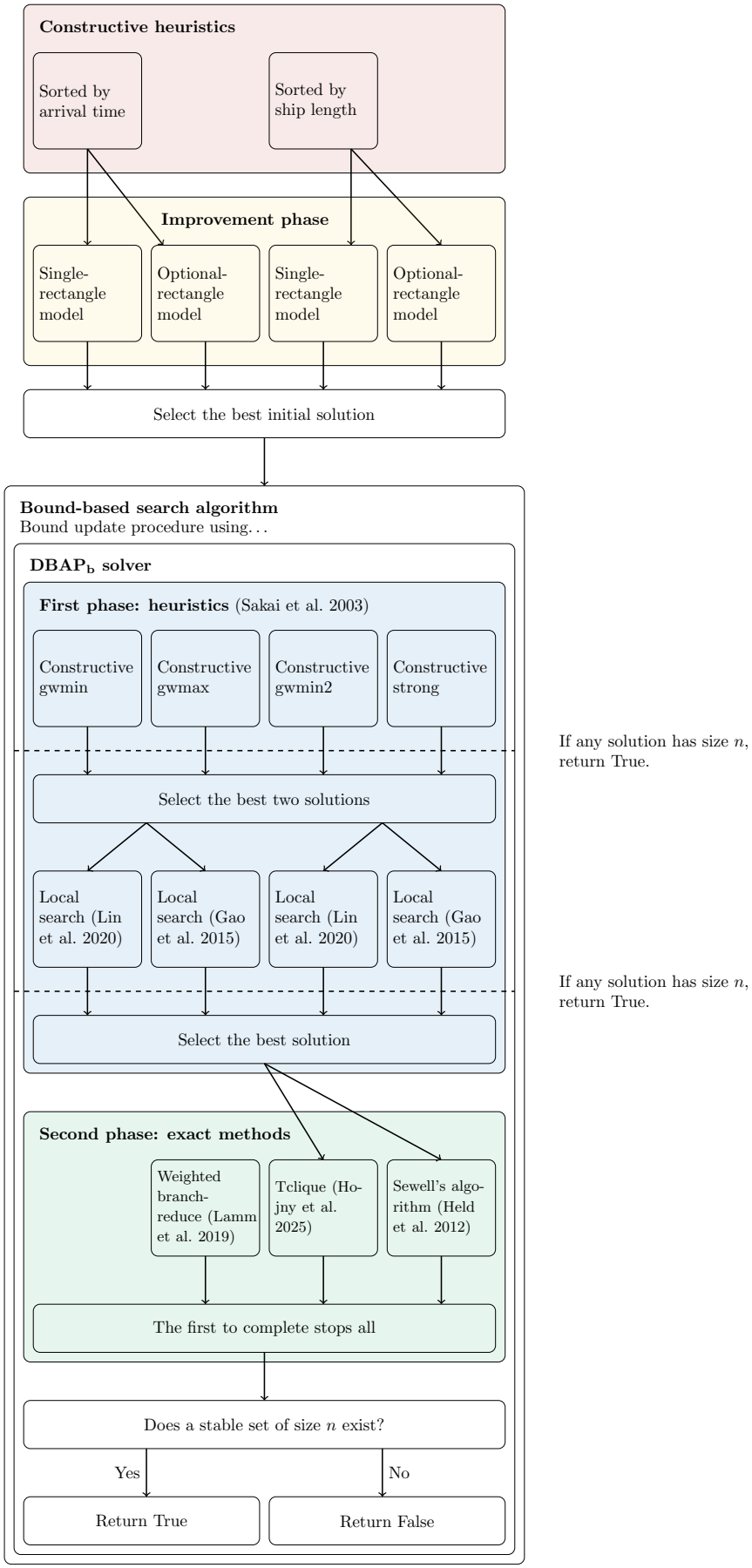


Figure 5: Schematic representation of the algorithm to solve $\mathrm{BAP_b}$, designed to use four parallel threads.

Let $T_v = \{t(v), \ldots, t(v) + h_{i(v),j(v)}\}$ be the set of time periods in which a ship is moored according to assignment $v \in V^{\mathrm{BAP}}$. Given a vector of non-negative Lagrangian multipliers $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_T)$, the modified cost of assignment v is

$$\tilde{c}_v(\boldsymbol{\pi}) = c_v + \ell_{i(v)} \sum_{t \in T_v} \pi_t.$$

The lowest modified cost associated with a ship i is $\tilde{m}_i(\boldsymbol{\pi}) = \min_{v \in V_i} \tilde{c}_v(\boldsymbol{\pi})$. The Lagrangian bound associated with multipliers $\boldsymbol{\pi}$ is

$$B(\boldsymbol{\pi}) = \sum_{i=1}^{n} \tilde{m}_i(\boldsymbol{\pi}) - L \sum_{t=1}^{T} \pi_t. \tag{4}$$

Let S be a feasible solution, and denote with

$$r_{St} = \sum_{\substack{v \in S \\ t \in T_v}} \ell_{i(v)}$$

the quay length occupied during period t according to S's berthing plan. Then, $B(\boldsymbol{\pi})$ is a valid lower bound on the cost of any feasible solution S:

$$\begin{aligned}
B(\boldsymbol{\pi}) &= \sum_{i=1}^{n} \tilde{m}_i(\boldsymbol{\pi}) - L \sum_{t=1}^{T} \pi_t = \\
&= \sum_{i=1}^{n} \min_{v \in V_i} \Big\{ c_v + \ell_{i(v)} \sum_{t \in T_v} \pi_t \Big\} - L \sum_{t=1}^{T} \pi_t \leq \\
&\leq \sum_{v \in S} c_v + \sum_{t=1}^{T} \underbrace{(r_{St} - L)}_{\leq 0} \pi_t \leq \\
&\leq \sum_{v \in S} c_v = U_S.
\end{aligned}$$

Bound $B(\boldsymbol{\pi})$ can be used to prune V^{BAP}. Indeed, a lower bound on the cost of any solution using assignment $v \in V^{\mathrm{BAP}}$ is

$$\tilde{c}_v(\boldsymbol{\pi}) + \sum_{i' \in \{1,\ldots,n\} \setminus \{i\}} \tilde{m}_{i'}(\boldsymbol{\pi}) - L \sum_{t=1}^{T} \pi_t = B(\boldsymbol{\pi}) + \tilde{c}_v(\boldsymbol{\pi}) - \tilde{m}_{i(v)}(\boldsymbol{\pi}).$$

If this cost is strictly larger than the cost U_S of an incumbent feasible solution, then v can be removed from V^{BAP}.

It remains to explain how to obtain good values for the Lagrangian multipliers $\boldsymbol{\pi}$. We start with $\boldsymbol{\pi} = \mathbf{0}$ and, at each iteration, we select the assignment with the smallest value of $\tilde{c}_v(\boldsymbol{\pi})$. Let r_t be the quay length occupied during period t according to the assignments selected so far. Then, $r_t - L$ is a subgradient direction for updating $\boldsymbol{\pi}$; we use Polyak (1969)'s method to determine the step size at each update.

Finally, we remark that this bound can be tightened by pricing the per-berth used capacity instead of the total capacity used at each period t. To this end, we use a much larger vector of lagrangian multipliers $\boldsymbol{\pi} = (\pi_{jt})_{j \in \{1,\ldots,m\}}^{t \in \{1,\ldots,T\}}$. The modified cost of an assignment v is

$$\tilde{c}_v(\boldsymbol{\pi}) = c_v + \sum_{(j,t) \in R_v} \pi_{jt},$$

where $R_v = \{j(v), \ldots, j(v) + \ell_{i(v)}\} \times \{t(v), \ldots, t(v) + h_{i(v),j(v)}\}$ is the discrete rectangle corresponding to assignment $v \in V^{\mathrm{BAP}}$. Redfining $\tilde{m}_i$ according to the new definition of $\tilde{c}_v$, the strengthened bound is

$$B(\boldsymbol{\pi}) = \sum_{i=1}^{n} \tilde{m}_i - \sum_{j=1}^{m} \sum_{t=1}^{T} \pi_{jt}.$$

In our implementation, we use this tighter version of the bound.

6.2.1 Solving the MWSSP

We solve the MWSSP in two phases. In the first phase, we run the four fast heuristic algorithms gwmin, gwmax, gwmin2, and strong (Sakai et al. 2003) that were also used in the DBAP_b approach described in Section 6.1.1. Whereas in Section 6.1.1 we set all weights to 1, here we use the actual weights p_v defined in Section 5. At the end, the best solution among those produced as described at the beginning of this section (under "Initial feasible solution" and "Initial solution improvement") and those produced by the heuristic MWSSP algorithms is retained as the incumbent. Unlike in the procedure described for the DBAP_b, we do not use the local search procedures of Lin et al. (2020) and Gao et al. (2015), because they only support the unweighted version of the problem.

In the second phase, we use exact solvers to search for the optimal MWSSP solution. We use the same three solvers described in Section 6.1.1, i.e., the weighted branch-and-reduce algorithm (Lamm et al. 2019), Tclique (Hojny et al. 2025), and Sewell's algorithm (Held et al. 2012). We pass the incumbent lower bound to all three algorithms; the branch-and-reduce and Sewell's algorithms additionally initialise the solver with the incumbent vertices, whereas Tclique's API accepts only the bound.

Figure 6 gives a schematic view of the overall algorithm used to solve DBAP_c.

7 Data

This section describes the data that was used for the computational experiments, which is based on real-world data from the port of Barcelona, Spain. We first describe the basic data generation procedure (Section 7.1) for problems with objective functions "max(compl)" and "$\sum$(compl)", spatial attribute "hyb", temporal aspect "dynamic", and handling attribute "fix". Then, in Section 7.2, we explain how to extend this method to objectives "$\sum w$(wait + hand)" and "$\sum w$(wait + tard)" and handling attribute "pos". Finally, in Section 7.3, we expand the instance set further by deriving, from each exsisting instance, new instances with different number of vessels. All instances and the best known solutions are available on GitHub (Santini 2026).

7.1 Base instance generation procedure

In this work, we introduce and make available several new sets of instances generated from real-world data. We consider two quays in Barcelona's port: 24B (APM Terminals) and 36A (BEST: Barcelona Europe South Terminal). We obtained data on ship movements at these two quays during 2024 from the Port of Barcelona through the open data initiative (Port de Barcelona 2024). Quay 24B is divided into 23 berths, and 36A into 31; however, analysing the data, we determined that only $m = 20$ berths are used in quay 24B and $m = 29$ in quay 36A. For both terminals, all berths have the same length λ.

For each ship i, the dataset indicates the leftmost (σ_i) and rightmost (τ_i) berths used by the ship. From these values, we compute the number of berths required to handle the ship as $\hat{\ell}_i = \tau_i - \sigma_i + 1$. The dataset also directly reports the ship length L_i in metres; however, for operational reasons, a ship can require more than $\lceil L_i/\lambda \rceil$ berths. Indeed, the data show that more than half of the ships require one additional berth, and, in some cases, two or even three extra berths are used. Therefore, we normalise the berth length as $\lambda = 1$ and use $\ell_i = \hat{\ell}_i$ as the ship length, because this quantity reflects the real number of berths used to handle the ship.

The data also record the periods $\underline{\theta}_i$ and $\bar{\theta}_i$ when the ship moored and left the quay. From this data, we compute a berth-independent handling time $h_i = \bar{\theta}_i - \underline{\theta}_i + 1$. The dataset does not contain the ship arrival times. If we used the mooring time as the arrival time ($a_i = \underline{\theta}_i$), it would be trivial to find the BAP optimal solution by using the real-life mooring time $\underline{\theta}_i$ and berth τ_i. Because such a solution was implemented in practice, it would be feasible, and because each ship moors at the earliest possible time, it would be optimal. Figure 7 shows two such berthing plans, relative to weeks six and seven of 2024, at the BEST terminal (quay 36A).

To obtain challenging instances, then, we proceed as follows. For each week w from the second to the 50th, we create an instance using data from weeks w and $w + 1$. We do not use the first and last weeks

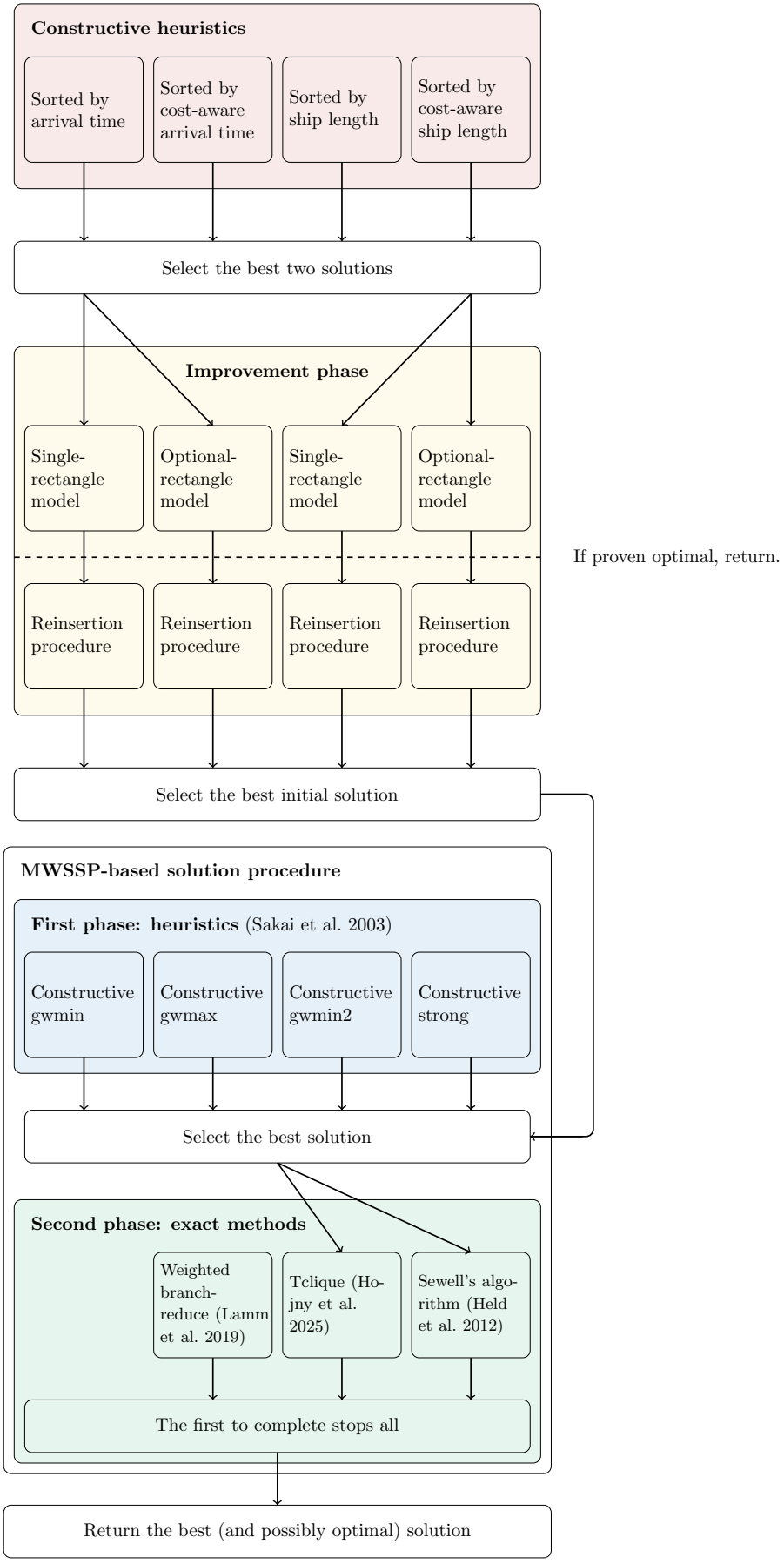


Figure 6: Schematic representation of the algorithm to solve $\mathrm{BAP_c}$, designed to use four parallel threads.

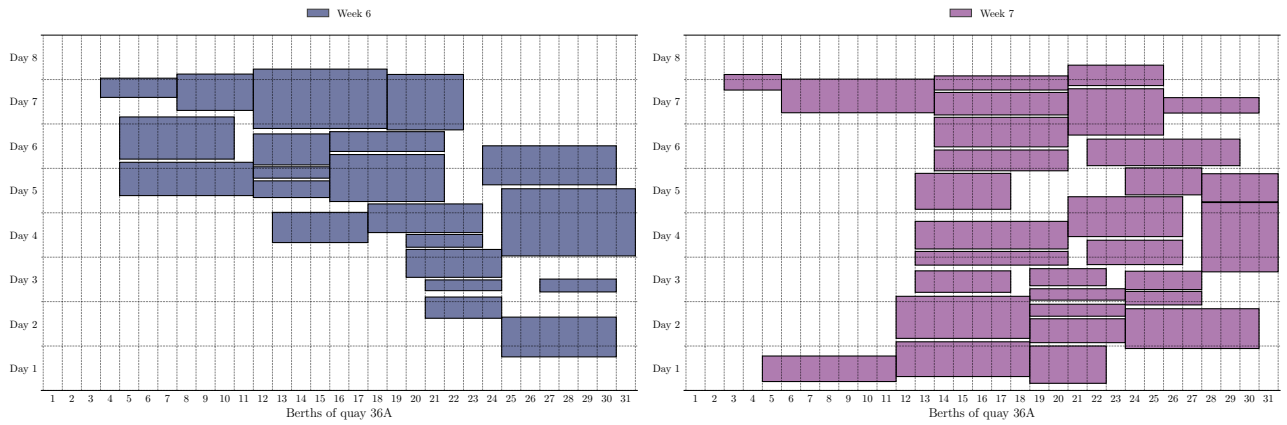


Figure 7: Real-life berthing plans for weeks six (left) and seven (right) of 2024 at the BEST terminal of Barcelona's port. Note that the two leftmost berths are always unused according to the data.

of the year to avoid spill-overs with years 2023 and 2025, respectively. In each instance, we use data from week w as-is, but we move backwards in time by one week the arrival times of all ships mooring during week $w + 1$. In this way, we create a busy week, with roughly twice as many ships as arrive at Barcelona's port during a typical week. Figure 8 shows the result of this procedure on the two weeks depicted in Figure 7. As shown by the many overlaps, a trivial berthing plan that allows each ship to berth as soon as it arrives would be infeasible for the generated instance. Because these instances correspond to artificially busy weeks, in Section 7.3, we will remove some of the port calls to create instances that more closely resemble the activity levels seen at the port of Barcelona in 2024.

The resulting base dataset contains one instance for each quay and each week from the second to the 50th, i.e., $2 \times 49 = 98$ instances. The corresponding problem types are "hyb|dyn|fix|max(compl),$\sum$(compl)". To ensure feasibility, we consider a time horizon of two weeks, discretised by the hour.

To summarise:

- We generate 98 base instances; 49 refer to quay 24B and 49 to quay 36A of the port of Barcelona, Spain.
- The 24B instances involve $m = 20$ berths and the 36A instances involve $m = 29$ berths.
- The 24B instances involve between $n = 12$ and $n = 29$ ships (median value: 24); the 36A instances involve between $n = 27$ and $n = 59$ ships (median value: 39).
- All instances contain $24 \times 14 = 336$ periods, i.e., a two-week time horizon discretised by the hour.

7.2 Instances for other problem variants

In this section, we explain how we expand the instance generation procedure to account for the other two cumulative objective functions of interest, and for position-dependent handling times. The objective functions "$\sum w(\text{wait} + \text{hand})$" and "$\sum w(\text{wait} + \text{tard})$" require ship-specific weights w_i; moreover, the latter objective function also requires desired ship completion times s_i.

With respect to ship weights, we use three alternative approaches. First, we remark that using uniform weights ($w_i = 1$ for all ships) does not reduce the objective functions to other objectives already considered. Therefore, the first approach consists in using uniform unitary weights for all ships. Because we do not have real-world data on ship priorities, in the second approach, we assign each ship a weight chosen uniformly at random among the integers from 1 to 10. In the third approach, we give to each ship one of two possible weights: 1 for a normal ship and 10 for a high-priority ship. We use ship length as a proxy for its possible importance. A ship in the top quartile by length has a 50% probability of being marked as high-priority; a ship in the bottom three quartiles by length only has a 10% probability of being marked as high-priority.

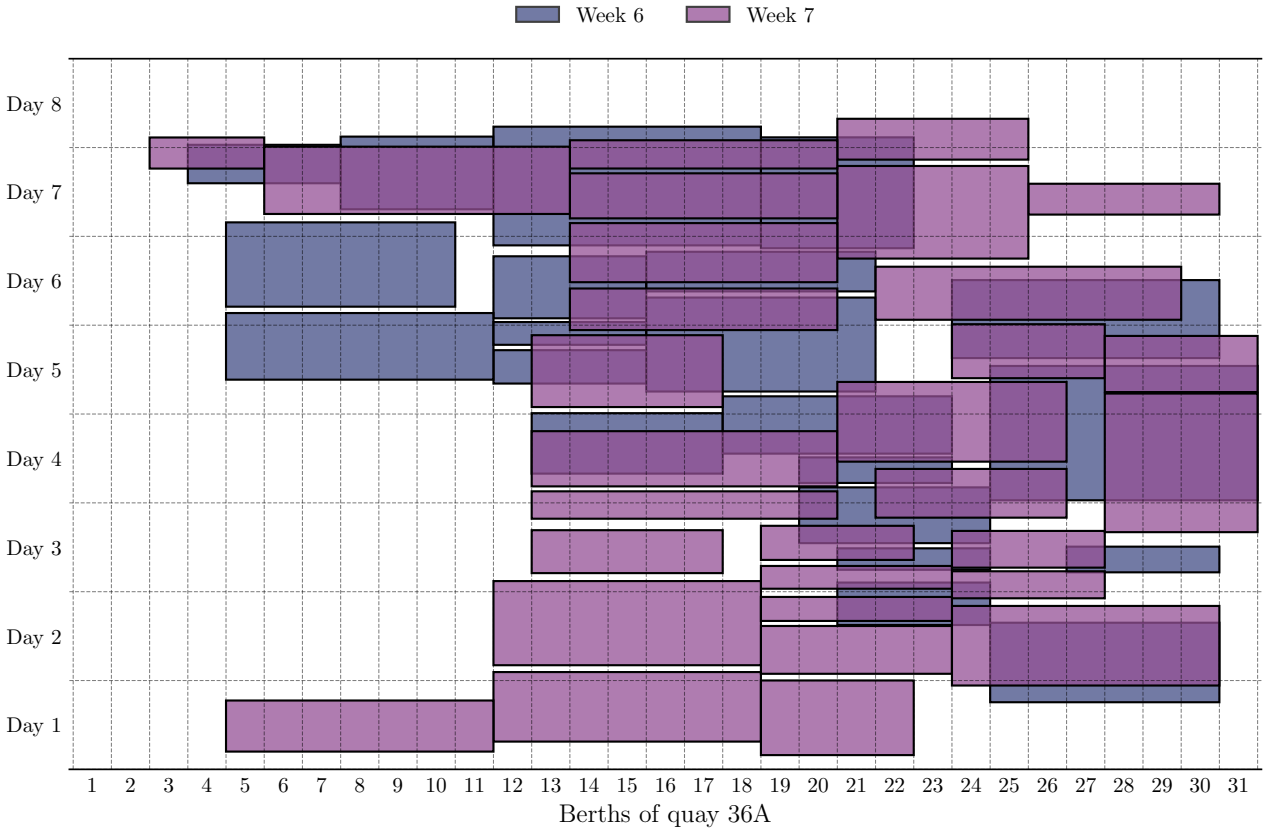


Figure 8: Instance resulting from considering weeks six and seven together, and moving the arrival times of the ships that moored during week seven backwards by one week. The resulting BAP instance can no longer be trivially solved to optimality by letting each ship berth as soon as it arrives.

Problem type	Weight generation method		Number of instances
hyb|dyn|fix|max(compl),$\sum$(compl)	—		98
hyb|dyn|pos|max(compl),$\sum$(compl)	—		98
hyb|dyn|fix|$\sum w$(wait + hand)	Constant	98	294
	Uniform	98	
	Normal/High-priority	98	
hyb|dyn|pos|$\sum w$(wait + hand)	Constant	98	294
	Uniform	98	
	Normal/High-priority	98	
hyb|dyn|fix|$\sum w$(wait + tard)	Constant	98	294
	Uniform	98	
	Normal/High-priority	98	
hyb|dyn|pos|$\sum w$(wait + tard)	Constant	98	294
	Uniform	98	
	Normal/High-priority	98	
Total			**1372**

Table 2: Summary table of the instance set reporting the number of instances generated for each problem type. Problems with opbjective functions "max(compl)" and "$\sum$(compl)" share the same instances.

Using the 98 base instances and the above three approaches to assign ship weights results in $98 \times 3 = 294$ additional instances for the "hyb|dyn|fix|$\sum w$(wait + hand)" problem.

With respect to ship desired completion times, again we could not find any real-world data and, therefore, we used the following instance generation procedure. First, remark that a desired completion time s_i cannot be smaller than $a_i + h_i$. In our procedure, the completion time is chosen uniformly at random in the interval $\{a_i + h_i, \ldots, a_i + 2h_i\}$.

Using the 98 base instances, the three approaches to assign ship weights, and the above approach to assign desired completion times results in $98 \times 3 \times 1 = 294$ additional instances for the "hyb|dyn|fix|$\sum w$(wait+ tard)" problem.

Finally, we generate instances for the problems with position-dependent handling times, i.e., h_{ij} rather than h_i. Based on a preliminary analysis of the data, we saw a pattern emerge especially in the 24B quay: ships moored towards either end of the quay had higher normalised handling times compared to ships mooring towards the centre of the quay. Because the handling time of a ship often increases with its size, the handling time is normalised by dividing it by the ship's length. The result of this analysis confirms the intuitive understanding that ships at one of the two ends of a quay have access to fewer cranes. In fact, cranes are mounted on rails and cannot cross each other. Therfore, a ship moored at the centre of a quay can use central cranes but also cranes to its immediate left and right; at the same time, it would block an idle crane to its left from reaching the rightmost end of the quay, where it could be used to speed up the loading and unloading operations of another ship moored there.

Based on this observation, we devised the following procedure for position-dependent handling times: if a ship's mooring position j implies that the ship overlaps with the 15% leftmost or 15% righmost berths, its handling time is $h_{ij} = \lceil 1.2 \cdot h_i \rceil$; otherwise, it is $h_{ij} = h_i$. In other words, we increase the handling time at peripheral berths by roughly 20%.

Using position-dependent handling times creates an additional variant for each of the $98+294+294 = 686$ instances created thus far, bringing the total number of instances in the dataset to 1372. Table 2 gives a summary of all instances generated accrding to the procedures described above.

7.3 Instances with different densities

The instances presented in Section 7.1 and Section 7.2 contain roughly twice as many ships as the port of Barcelona receives in a typical week. In this section, we present instances derived from the one described above, but with up to 50% fewer ships, to better match the typical activity level seen at a mid-large container port such as that of Barcelona, Spain. These instances will also allow for an experimental analysis to uncover the relationship between instance "density" and computational difficulty. In particular, we select two representative objective functions, "hyb|dyn|fix|max(compl)" for a bottleneck objective and "hyb|dyn|pos|$\sum$(compl)" for a cumulative objective, and the "fix" handling time attribute, and apply the following procedure to the base instances matching these attributes. We create five additional instances for each existing one, in which we retain respectively 90%, 80%, 70%, 60%, and 50% of the ships, selected uniformly at random. All other attributes, including the time discretisation and the number of berths are kept fixed. This procedure generates $5 \times 98 = 490$ additional instances.

8 Results

This section presents the results obtained by the five MIP models and by our proposed approaches, for the BAP_b and the BAP_c. After describing the computational environment in Section 8.1, we present the results for the BAP_b in Section 8.2 and those for the BAP_c in Section 8.3. Section 8.4 further explores the impact of instance density on the algorithms' performance.

Throughout the rest of this section, our tables describe the following metrics:

- *Num feasible* reports the number of instances, in each group of 98, for which a solver has found at least one feasible solution.
- *Num optimal* reports the number of instances, in each group of 98, for which the solver produced an optimal solution and proved its optimality.
- *Avg % gap* reports the average (over each group of 98 instances) percent optimality gap between the tightest upper and lower bounds produced by the solver. Instances that did not produce any feasible solution get assigned a percent gap of 100%.
- *Avg runtime (s)* is the average (over each group of 98 instances) total wallclock runtime. For the MIPs, this includes the time to build the models before the black-box solver is invoked. For our proposed approaches, it includes the time necesary to build the graphs.

Bold numbers in the rows relative to the last four metrics highlight the approach that produced the best value.

8.1 Computational environment

We ran our computational campaign on a cluster with Intel Xeon processors running at 3.4GHz and 128GB of memory. All approaches were allocated a wall-clock time limit of one hour. The limit includes the time required to build the model and to solve it (for the MIPs), and the time to create the graphs and run all phases of the algorithms (for our proposed approaches). The following software was used: 1. The MIP models RP, PA, GSPP, S, and TI were solved using Gurobi 13.0.3. The solver was allowed to use four concurrent threads. 2. The solvers presented in Section 4 and Section 5 were implemented in C++. The implementation allows the use of up to four concurrent threads, as shown in Figure 5 and Figure 6. 3. The Constraint Programming model used in the imporvement phases was solved with CP-SAT (Perron et al. 2023), part of the OR-Tools suite, version 9.15. 4. The four constructive heuristics gwmin, gwmax, gwmax2, and strong are used through the StableSolver library (Fontan 2020), commit `efab011`. 5. The local search algorithm (Lin et al. 2020) is also used through the StableSolver library. 6. The branch-and-reduce algorithm is used through the KaMIS library (Lamm et al. 2019), version 3.2. 7. The Tclique algorithm has been extracted from the implementation shipped with the

Metric	Handling	RP	PA	GSPP	S	TI	Our method
Num feasible	fix	98	98	98	98	98	98
	pos	98	98	98	98	76	98
Num optimal	fix	67	57	68	68	62	**74**
	pos	46	43	51	46	34	**56**
Avg % gap	fix	2.46	5.78	5.14	2.36	4.41	**1.85**
	pos	5.57	10.73	7.78	5.43	27.86	**3.23**
Avg runtime (s)	fix	1183.4	1979.3	1345.2	1134.4	1605.7	**971.6**
	pos	1990.8	2510.2	1970.4	2007.7	2632.6	**1625.8**

Table 3: Summary of the results for the BAP_b with objective "max(compl)".

Metric	Handling	RP	PA	GSPP	S	TI	Our method
Num feasible	fix	98	98	98	98	98	98
	pos	98	98	98	98	88	98
Num optimal	fix	29	41	46	29	**71**	33
	pos	15	31	37	14	17	**42**
Avg % gap	fix	3.48	10.22	1.16	3.40	**0.45**	4.62
	pos	6.18	10.56	**1.63**	6.11	13.45	2.36
Avg runtime (s)	fix	2596.4	2454.9	2146.6	2592.7	**1168.2**	2333.7
	pos	3099.0	2733.0	2549.1	3102.9	3139.8	**2372.1**

Table 4: Summary of the results for the BAP_c with objective "$\sum$(compl)".

SCIP solver (Hojny et al. 2025), version 10.0.3. 8. Sewell's algorithm is used through the exactcolors library (Held et al. 2012), commit `5dd66d0`.

8.2 BAP_b results

Table 3 shows the results on the BAP_b instances with objective function "max(compl)". All methods except the TI formulation find at least one primal feasible solution for all instances. The instances with position-based handling time appear considerably harder than those for which the handling times do not depend on the berthing position. Our method, described in Section 4 and Section 6.1, finds 74 out of 98 proven optimal values for the "fix" instances and 56 out of 98 for the "pos" instances. By contrast, the next-best methods are models GSPP and S for the "fix" instances, which find 68 optimal solution, and model GSPP for the "pos" instances, which finds 51 optima.

Our method also produces the lowest average gaps (1.58% for the "fix" instances, and 3.23% for the "pos" instances). Interestingly, the second-lowest average gaps are produced by formulation S, whose gaps are lower than those produced by formulation GSPP. This latter formulation is known to have a tight continuous realaxation and, therefore, produce small optimality gaps. However, its effectiveness has been tested in the literature on cumulative objective functions only; our experiments show that the formulation is not as effective for bottleneck objective functions.

At the other hand of the spectrum, the formulations that find the fewest optima are PA for the "fix" instances and TI for the "pos" instances.

8.3 BAP_c results

Tables 4 to 6 report the results on the BAP_c instances, respectively for objective function "$\sum$(compl)", "$\sum w$(wait+hand)", and "$\sum w$(wait+tard)". In all three cases, our method (described in Section 5 and

Metric	Weights type	Handling	RP	PA	GSPP	S	TI	Our method
Num feasible	Constant	fix	98	98	98	98	98	98
		pos	98	98	98	98	87	98
	Uniform	fix	98	98	98	98	98	98
		pos	98	98	98	98	90	98
	Normal/High-priority	fix	98	98	98	98	98	98
		pos	98	98	98	98	90	98
Num optimal	Constant	fix	30	42	44	31	**73**	44
		pos	14	31	36	14	17	**44**
	Uniform	fix	36	52	57	37	**78**	54
		pos	16	43	45	16	21	**57**
	Normal/High-priority	fix	31	46	51	32	**76**	51
		pos	18	44	55	18	19	**63**
Avg % gap	Constant	fix	13.52	20.74	4.69	13.30	**1.69**	5.67
		pos	21.50	21.17	6.17	21.68	22.50	**5.33**
	Uniform	fix	8.34	17.10	2.61	8.07	**1.71**	4.25
		pos	14.90	18.17	3.59	15.23	16.76	**3.31**
	Normal/High-priority	fix	6.36	17.29	1.84	6.34	**1.35**	2.51
		pos	10.97	17.16	**1.61**	11.09	15.46	2.14
Avg runtime (s)	Constant	fix	2607.3	2439.9	2194.5	2575.3	**1113.7**	2017.3
		pos	3141.7	2736.6	2516.9	3111.4	3099.9	**2190.1**
	Uniform	fix	2400.1	2158.1	1816.8	2389.6	**911.0**	1743.3
		pos	3093.4	2493.7	2221.6	3097.3	3068.3	**1779.8**
	Normal/High-priority	fix	2555.8	2307.2	1974.4	2523.2	**993.8**	1888.0
		pos	3023.8	2407.1	1939.8	3009.4	3042.5	**1614.4**

Table 5: Summary of the results for the BAP_c with objective "$\sum w(\text{wait} + \text{hand})$".

Metric	Weights type	Handling	RP	PA	GSPP	S	TI	Our method
Num feasible	Constant	fix	98	98	98	98	98	98
		pos	98	98	98	98	86	98
	Uniform	fix	98	98	98	98	98	98
		pos	98	98	98	98	91	98
	Normal/High-priority	fix	98	98	98	98	98	98
		pos	98	98	98	98	90	98
Num optimal	Constant	fix	30	41	47	31	**67**	42
		pos	19	31	41	21	30	**43**
	Uniform	fix	34	50	57	35	**79**	52
		pos	24	40	51	21	31	**52**
	Normal/High-priority	fix	32	48	51	34	**72**	51
		pos	25	39	46	22	32	**48**
Avg % gap	Constant	fix	55.29	34.03	19.39	52.41	**6.23**	26.98
		pos	64.62	40.16	**20.55**	63.55	37.12	32.13
	Uniform	fix	50.23	27.77	15.98	48.68	**6.85**	22.13
		pos	60.29	33.30	**18.16**	59.49	32.30	30.44
	Normal/High-priority	fix	51.99	28.07	15.39	50.10	**6.25**	20.87
		pos	59.84	31.71	**16.41**	60.17	34.84	26.99
Avg runtime (s)	Constant	fix	2584.1	2429.9	2090.8	2560.8	**1229.3**	2136.0
		pos	2949.4	2742.2	2453.2	2900.6	2679.6	**2223.7**
	Uniform	fix	2454.3	2249.0	1853.0	2421.5	**951.2**	1806.7
		pos	2864.1	2566.9	2055.2	2906.1	2600.4	**1922.4**
	Normal/High-priority	fix	2525.7	2222.0	1915.6	2483.8	**1108.4**	1891.4
		pos	2846.3	2530.9	2193.2	2854.0	2601.1	**2081.9**

Table 6: Summary of the results for the BAP_c with objective "$\sum w(\text{wait} + \text{tard})$".

Section 6.2) is the best-performing on the "pos" instances, but not on the "fix" instances. Regarding the "fix" instances, the TI formulation finds the most optima, has the lowest average gaps and uses less time in seven out of seven groups of instances across the three objective functions. The GSPP formulation is consistently the second-best performing formulation for these instances. With respect to the "pos" instances, our method finds the most optima and has the fastest runtimes in seven out of seven groups of instances across the three objective functions, and yields the lowest average gap in six out of seven groups. The GSPP formulation yields the lowest average gaps in the remaining group, i.e., for the "$\sum w(\text{wait} + \text{hand})$" objective function, with normal/high-priority weights.

Position-dependent handling times make the problems with cumulative objectives harder to solve with the MIP formulations (the same effect is observed for bottleneck objective functions in Section 8.2). Often, the "pos" instances have fewer proven optima, larger average gaps, and longer solution times, irrespective of the formulation used to solve them.

Conversely, for some combination of objective function and weight type, our method solves more "pos" than "fix" instances. For the "$\sum(\text{compl})$" objective, we solve 33 "fix" instances to optimality and 42 "pos" instances. For the "$\sum w(\text{wait} + \text{hand})$" objective, the largest jump is observed for instances with the normal/high-priority weights: we solve to optimality 51 "fix" instances and 63 "pos" instances. This difference is less marked for the "$\sum w(\text{wait} + \text{tard})$" objective function, where the number of "fix" and "pos" instances solved to optimality by our method is roughly the same. A plausible explanation is that position-dependent handling times differentiate the cost of assigning ships to berths, potentially improving the effectiveness of our cost-based graph pruning. For the "$\sum(\text{compl})$" and "$\sum w(\text{wait} + \text{hand})$", the value of the handling time appears directly in the objective value of the schedules (see (1a) and (1b)). Therefore, the increased cost of assigning a ship to a berth with a high handling time h_{ij} has a direct impact on the resulting schedule cost and intervenes directly in the graph pruning techniques we use. Conversely, in the "$\sum w(\text{wait} + \text{tard})$", a longer handling time changes the objective function only if it increases a ship's tardiness. Therefore, its impact in a schedule's cost (see (1c)) and in the pruning technique is more indirect. These observations might also explain why the "pos" instances are harder than the "fix" instances in the BAP_b case: the pruning techniques for the BAP_b are less sophisticated than those for the BAP_c, and do not exploit the extra information conveyed by berth-dependent handling times.

8.4 Impact of instance densities

Tables 7 and 8 report the results on, respectively, the BAP_b and BAP_c instances with reduced densities described in Section 7.3. In both cases, reducing the density (i.e., the number of ships arriving during the same planning horizon) results in an easier problem, as fewer and fewer conflicts arise and ships are increasingly able to moor at convenient times and locations. For both problem types, the methods that outperform the others on the full instances are also ranked first on the instances with lower densities, thus showing that the results presented in Sections 8.2 and 8.3 are robust to density.

Regarding the BAP_b with "max(compl)" objective and "fix" handling attribute, our method generally outperforms the others in terms of number of optimal solutions found, average gaps and solution times. For instances in which we retain 50% and 60% of the ships, our method terminates almost instantly because the initial heuristics find the optimal solution, thus considerably reducing the range of values the bound-based algorithm considers. Furthermore, instances with few ships result in sparse graphs G_M^{BAP} and easy MSSP problems. The runtime abruptly increase starting from instances in which we retain 70% of the ships, but are still shorter than for the instances in which all ships are retained.

Regarding the BAP_c with "sum(compl)" objective and "fix" handling attribute, a similar pattern emerges with the least dense instances being solved in a few seconds by the best-performing method (the TI formulation). Removing just 10% of the ships (i.e., in the instances with 90% density) causes a sharp speed-up of this formulation, from 1168 to 692 seconds. The other formulations are also faster when fewer ships are involved, but the speed-up is not as drastic; only our method experiences a comparative speed-up in relative terms, from 2334 to 1234 seconds. Still, the TI formulation is the only method that solves to optimality all instances up to 70% density and, even for instances with 50%

Metric	Density	RP	PA	GSPP	S	TI	Our method
Num optimal	50	**98**	**98**	97	**98**	**98**	**98**
	60	**98**	97	**98**	**98**	**98**	**98**
	70	**97**	96	**97**	**97**	**97**	**97**
	80	**94**	88	93	93	93	**94**
	90	79	74	80	81	77	**87**
	100	67	57	68	68	62	**74**
Avg % gap	50	**0**	**0**	0.03	**0**	**0**	**0**
	60	**0**	0.03	**0**	**0**	**0**	**0**
	70	0.05	0.30	**0.03**	0.07	0.08	**0.03**
	80	0.15	0.47	0.28	0.15	0.23	**0.09**
	90	0.82	1.99	1.69	0.76	1.71	**0.37**
	100	2.46	5.78	5.14	2.36	4.41	**1.85**
Avg runtime (s)	50	0.14	190.8	73.20	2.25	12.81	**0.05**
	60	**0.59**	296.3	58.20	0.90	38.60	0.80
	70	41.27	468.5	110.0	40.56	144.0	**37.35**
	80	193.5	957.2	353.9	192.5	413.3	**149.6**
	90	730.3	1507.7	874.4	693.3	1055.6	**473.2**
	100	1183.4	1979.3	1345.2	1134.4	1605.7	**971.6**

Table 7: Summary of the results for the BAP_b on instances with different densities. The objective function is "max(compl)" and the handling type is "fix" for all instances.

Metric	Density	RP	PA	GSPP	S	TI	Our method
Num optimal	50	97	97	**98**	97	**98**	97
	60	93	95	96	92	**98**	96
	70	77	85	82	80	**98**	91
	80	68	70	74	68	**95**	85
	90	44	58	58	46	**83**	66
	100	29	41	46	29	**71**	33
Avg % gap	50	0.02	0.00	0.00	0.01	**0**	0.02
	60	0.05	0.01	0.01	0.05	**0**	0.04
	70	0.36	0.12	0.08	0.31	**0**	0.19
	80	0.91	0.39	0.25	0.88	**0.02**	0.26
	90	2.05	2.42	0.71	1.98	**0.21**	0.97
	100	3.48	10.22	1.16	3.40	**0.45**	4.62
Avg runtime (s)	50	40.41	106.1	73.12	40.95	**2.59**	36.78
	60	222.0	235.2	195.8	232.6	**4.68**	73.73
	70	849.9	745.3	732.8	768.6	**18.79**	267.8
	80	1196.4	1265.1	1168.1	1199.3	**199.5**	531.4
	90	2112.9	1825.1	1633.8	2075.2	**692.1**	1233.5
	100	2596.4	2454.9	2146.6	2592.7	**1168.2**	2333.7

Table 8: Summary of the results for the BAP_c on instances with different densities. The objective function is "sum(compl)" and the handling type is "fix" for all instances.

density, only the TI and the GSPP formulations solve all instances to optimality. This shows that, for cumulative objective functions, instances with roughly as many ships per week as really arrive in a medium-sized port such as Barcelona still need advanced decision-support tools to make provably optimal decisions.

9 Conclusions

We developed two exact approaches to the berth allocation problem based on a conflict graph in which vertices represent ship assignments and edges represent incompatible pairs of assignments. We classified some of the most commonly used objective functions into two large families, bottleneck and cumulative. For bottleneck objectives, the decision version of the BAP can be transformed into an unweighted maximum stable set problem. We then combinined this transformation with a search algorithm that tightens bounds on the optimum and refines the time discretisation as needed. For cumulative objectives, the BAP can be transformed into a single maximum weighted stable set problem, and bounds derived from feasible schedules and Lagrangian relaxations allowed us to prune the corresponding graph.

We compared the two approaches with five mixed-integer programming formulations on publicly available benchmark instances derived from 2024 ship movements at two container terminals in Barcelona. Within the one-hour time limit, the proposed algorithm for bottlenck objectives solved more instances to optimality than any of the five formulations and achieved the smallest average optimality gaps with both berth-independent and berth-dependent handling times. For cumulative objectives with berth-dependent handling times, the proposed method solved the most instances to optimality and had the shortest average runtime in all instance groups. A generalised set partitioning formulation achieved the smallest average gap in five of these groups, including all three groups in which the objective function minimises the weighted waiting and tardiness. For cumulative objectives with berth-independent handling times, a time-index formulation led on all three measures in every instance group. Reducing the number of ships generally made the tested problems easier to solve for all methods without considerably changing their relative rankings.

The remaining large gaps for the objective function involving tardiness motivate investigating whether better bounds can help prune the weighted conflict graph. In particuar, the generalised set partitioning relaxation enjoys considerably tighter bounds than the other methods. Therefore, future research might explore ways to transform bounds from this formulation into graph pruning rules. For longer planning horizons, another direction is to decompose the conflict graph into subgraphs associated with different time windows, and coordinate ship assignments and berth conflicts across these windows.

Statements and declarations

Acknowledgements. The author is extremely grateful to two anonymous reviewers and to the editor who, with their extensive comments, have prompted major improvements in the present work.

Funding. This work was supported by MICIU/AEI/10.13039/501100011033 (Ministerio de Ciencia, Innovación y Universidades; Agencia Estatal de Investigación) and the FSE+ (Fundo Social Europeo+) via grant RYC2022-035269-I.

Competing interests. The author has no competing interests to declare that are relevant to the content of this article.

Data availability. The data used for this study is available on GitHub (Santini 2026).

No other statements are applicable to this work.

References

Bierwirth, Christian and Frank Meisel (2015). "A follow-up survey of berth allocation and quay crane scheduling problems in container terminals". In: *European Journal of Operational Research* 244, pp. 675–689. DOI: 10.1016/j.ejor.2014.12.030.

Błażewicz, Jacek, Tai Chiu Cheng Cheng, Maciej Machowiak, and Ceyda Oğuz (2011). "Berth and quay crane allocation: a moldable task scheduling model". In: *Journal of the Operational Research Society* 62 (7), pp. 1189–1197. DOI: 10.1057/jors.2010.54.

Buhrkal, Katja, Sara Zuglian, Stefan Ropke, Jesper Larsen, and Richard Lusby (2011). "Models for the discrete berth allocation problem: A computational comparison". In: *Transportation Research Part E: Logistics and Transportation Review* 47 (4), pp. 461–473. DOI: 10.1016/j.tre.2010.11.016.

Cahyono, Rully Tri, Saskia Puspa Kenaka, and Bayu Jayawardhana (2022). "Simultaneous Allocation and Scheduling of Quay Cranes, Yard Cranes, and Trucks in Dynamical Integrated Container Terminal Operations". In: *IEEE Transactions on Intelligent Transportation Systems* 23 (7), pp. 8564–8578. DOI: 10.1109/TITS.2021.3083598.

Cheong, Chun Yew, Kay Chen Tan, Dikai Liu, and Chiuhsiang Joe Lin (2010). "Multi-objective and prioritized berth allocation in container ports". In: *Annals of Operations Research* 180, pp. 63–103. DOI: 10.1007/s10479-008-0493-0.

Christensen, Clement Gram and Cecilie Terese Holst (2008). "Allokering af kajplads i containerhavne". Thesis number Imm-m.sc.-2008-37. MA thesis. Danish Technical University. URL: https://findit.dtu.dk/en/catalog/5a02e31d5010df4c820fe078.

Cornaz, Denis, Fabio Furini, Enrico Malaguti, and Alberto Santini (2019). "A note on Selective line-graphs and partition colorings". In: *Operations Research Letters* 47 (6), pp. 565–568. DOI: 10.1016/j.orl.2019.08.005.

Correcher, Juan Francisco, Thomas Van den Bossche, Ramon Alvarez-Valdes, and Greet Vanden Berghe (2019). "The berth allocation problem in terminals with irregular layouts". In: *European Journal of Operational Research* 272 (3), pp. 1096–1108. DOI: 10.1016/j.ejor.2018.07.019.

Demange, Marc, Tınaz Ekim, Bernard Ries, and Cerasela Tanasescu (2015). "On some applications of the selective graph coloring problem". In: *European Journal of Operational Research* 240 (2), pp. 307–314. DOI: 10.1016/j.ejor.2014.05.011.

Emde, Simon, Nils Boysen, and Dirk Briskorn (2014). "The berth allocation problem with mobile quay walls: problem definition, solution procedures, and extensions". In: *Journal of Scheduling* 17, pp. 289–303. DOI: 10.1007/s10951-013-0358-5.

Ernst, Andreas, Ceyda Oğuz, Gaurav Singh, and Gita Taherkhani (2017). "Mathematical models for the berth allocation problem in dry bulk terminals". In: *Journal of Scheduling* 20, pp. 459–473. DOI: 10.1007/s10951-015-0510-8.

Fontan, Florian (Aug. 29, 2020). *A solver for the maximum(-weight) independent set and the maximum(-weight) clique problems*. URL: https://github.com/fontanf/stablesolver (visited on 07/21/2026).

Furini, Fabio, Enrico Malaguti, and Alberto Santini (2018). "An exact algorithm for the Partition Coloring Problem". In: *Computers & Operations Research* 92, pp. 170–181. DOI: 10.1016/j.cor.2017.12.019.

Gao, Chao, Xin Yao, Thomas Weise, and Jinlong Li (2015). "An efficient local search heuristic with row weighting for the unicost set covering problem". In: *European Journal of Operational Research* 246.3, pp. 750–761. DOI: 10.1016/j.ejor.2015.05.038.

Grubišić, Neven and Mate Barić (2015). "A Contribution to Berth Allocation Problem Solution with Draft Restrictions". In: *Pomorski zbornik* 49–50 (1). Article ID 138194, pp. 127–142.

Guan, Yongpei and Raymond Kaman Cheung (2004). "The berth allocation problem: models and solution methods". In: *OR Spectrum* 26, pp. 75–92. DOI: 10.1007/s00291-003-0140-8.

Hansen, Pierre and Ceyda Oğuz (May 2003). *A Note on Formulations of Static and Dynamic Berth Allocation Problems*. Tech. rep. G-2003-30. Montréal, Canada: Le Cahiers du GERAD.

Held, Stephan, William Cook, and Edward Clyde Sewell (2012). "Maximum-weight stable sets and safe lower bounds for graph coloring". In: *Mathematical Programming Computation* 4, pp. 363–381. DOI: 10.1007/s12532-012-0042-3.

Hojny, Christopher et al. (2025). *The SCIP Optimization Suite 10.0*. arXiv: 2511.18580 [math.OC]. URL: https://arxiv.org/abs/2511.18580.

Hoshino, Edna Ayako, Yuri Abitbol Frota, and Cid Carvalho de Souza (2011). "A branch-and-price approach for the partition coloring problem". In: *Operations Research Letters* 39 (2), pp. 132–137. DOI: 10.1016/j.orl.2011.02.006.

Imai, Akio, Etsuko Nishimura, and Stratos Papadimitriou (2001). "The dynamic berth allocation problem for a container port". In: *Transportation Research Part B: Methodological* 35 (4), pp. 401–417. DOI: 10.1016/S0191-2615(99)00057-0.

Ji, Bin, Han Huang, and Samson Yu (2023). "An Enhanced NSGA-II for Solving Berth Allocation and Quay Crane Assignment Problem With Stochastic Arrival Times". In: *IEEE Transactions on Intelligent Transportation Systems* 24 (1), pp. 459–473. DOI: 10.1109/TITS.2022.3213834.

Kameshwaran, Sampath, Alfiya Tezabwala, and Vinayaka Pandit (2013). "Constraint based Berth Allocation and Ship Scheduling with Upstream Supply Planning". In: *Proceedings of the 7th Scheduling and Planning Applications Workshop (SPARK 2013)* (June 11, 2013). Ed. by Luis Castillo Vidal, Steve Chien, and Riccardo Rasconi. Rome, Italy, pp. 77–83. URL: https://icaps13.icaps-conference.org/wp-content/uploads/2013/05/spark13-proceedings.pdf.

Kramer, Arthur, Eduardo Lalla-Ruiz, Manuel Iori, and Stefan Voß (2019). "Novel formulations and modeling enhancements for the dynamic berth allocation problem". In: *European Journal of Operational Research* 278 (1), pp. 170–185. DOI: 10.1016/j.ejor.2019.03.036.

Lalla-Ruiz, Eduardo, Christopher Expósito-Izquierdo, Belén Melián-Batista, and José Marcos Moreno-Vega (2016). "A Set-Partitioning-based model for the Berth Allocation Problem under Time-Dependent Limitations". In: *European Journal of Operational Research* 250 (3), pp. 1001–1012. DOI: 10.1016/j.ejor.2015.10.021.

Lalla-Ruiz, Eduardo, Belén Melián-Batista, and José Marcos Moreno-Vega (2012). "Artificial intelligence hybrid heuristic based on tabu search for the dynamic berth allocation problem". In: *Engineering Applications of Artificial Intelligence* 25 (6), pp. 1132–1141. DOI: 10.1016/j.engappai.2012.06.001.

Lamm, Sebastian, Christian Schulz, Darren Strash, Robert Williger, and Huashuo Zhang (2019). "Exactly Solving the Maximum Weight Independent Set Problem on Large Real-World Graphs". In: *2019 Proceedings of the Meeting on Algorithm Engineering and Experiments (ALENEX)* (Jan. 7–9, 2019). Ed. by Stephen Kobourov and Henning Meyerhenke. San Diego, United States of America, pp. 144–158. DOI: 10.1137/1.9781611975499.12.

Lee, Der-Horng and Jiang Hang Chen (2010a). "An Improved Approach for Quay Crane Scheduling with Non-Crossing Constraints". In: *Engineering Optimization* 42 (1), pp. 1–15. DOI: 10.1080/03052150902943020.

Lee, Der-Horng and Hui Qiu Wang (2010b). "Integrated discrete berth allocation and quay crane scheduling in port container terminals". In: *Engineering Optimization* 42.8, pp. 747–761. DOI: 10.1080/03052150903406571.

Li, Chung-Lun, Xiaoqiang Cai, and Chung-Yee Lee (1998). "Scheduling with multiple-job-on-one-processor pattern". In: *IIE Transactions* 30.5, pp. 433–445. DOI: 10.1080/07408179808966484.

Li, Ying, Feng Chu, Feifeng Zheng, and Ming Liu (2022). "A Bi-Objective Optimization for Integrated Berth Allocation and Quay Crane Assignment With Preventive Maintenance Activities". In: *IEEE Transactions on Intelligent Transportation Systems* 23 (4), pp. 2938–2955. DOI: 10.1109/TITS.2020.3023701.

Lim, Andrew (1988). "The berth planning problem". In: *Operations Research Letters* 22, pp. 105–110. DOI: 10.1016/s0167-6377(98)00010-8.

Lin, Weibo, Fuda Ma, Zhouxing Su, Qingyun Zhang, Chumin Li, and Zhipeng Lü (2020). "Weighting-based parallel local search for optimal camera placement and unicost set covering". In: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion* (July 8–12, 2020). Ed. by Carlos Artemio Coello Coello. GECCO '20. Cancún, Mexico: Association for Computing Machinery, pp. 3–4. ISBN: 9781450371278. DOI: 10.1145/3377929.3398184.

Liu, Ming, Rongfan Liu, Feng Chu, and Chengbin Chu (July 2018). "Mathematical Model and Solution Approach for Berth Allocation Problem in Tidal Bulk Ports with Different Vessel Draft

Requirements". In: *Proceedings of the IEEE 15th International Conference on Service Systems and Service Management*. Hangzhou, China, pp. 1–6. DOI: 10.1109/ICSSSM.2018.8465036.

Malaguti, Enrico, Silvano Martello, and Alberto Santini (2018). "The traveling salesman problem with pickups, deliveries, and draft limits". In: *Omega* 74, pp. 50–58. DOI: 10.1016/j.omega.2017.01.005.

Monaco, Maria Flavia and Marcello Samarra (2007). "The Berth Allocation Problem: A Strong Formulation Solved by a Lagrangean Approach". In: *Transportation Science* 41 (2), pp. 265–280. DOI: 10.1287/trsc.1060.0171.

Nishi, Tatsushi, Tatsuya Okura, Eduardo Lalla-Ruiz, and Stefan Voß (2020). "A dynamic programming-based matheuristic for the dynamic berth allocation problem". In: *Annals of Operations Research* 286, pp. 391–410. DOI: 10.1007/s10479-017-2715-9.

Perron, Laurent, Frédéric Didier, and Steven Gay (2023). "The CP-SAT-LP Solver". In: *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)* (July 27–31, 2023). Ed. by Roland H. C. Yap. Vol. 280. Leibniz International Proceedings in Informatics (LIPIcs). Toronto, Canada: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, pp. 1–2. ISBN: 978-3-95977-300-3. DOI: 10.4230/LIPIcs.CP.2023.3.

Polyak, Boris Teodorovich (1969). "Minimization of unsmooth functionals". In: *USSR Computational Mathematics and Mathematical Physics* 9.3, pp. 14–29. DOI: 10.1016/0041-5553(69)90061-5.

Port de Barcelona (2024). *Open Data*. URL: https://opendata.portdebarcelona.cat/en/.

Qin, Tianbao, Yuquan Du, and Mei Sha (2016). "Evaluating the solution performance of IP and CP for berth allocation with time-varying water depth". In: *Transportation Research Part E: Logistics and Transportation Review* 87, pp. 167–185. DOI: 10.1016/j.tre.2016.01.007.

Rashidi, Hassan and Edward P. K. Tsang (2013). "Novel constraints satisfaction models for optimization problems in container terminals". In: *Applied Mathematical Modelling* 37.6, pp. 3601–3634. DOI: 10.1016/j.apm.2012.07.042.

Sakai, Shuichi, Mitsunori Togasaki, and Koichi Yamazaki (2003). "A note on greedy algorithms for the maximum weighted independent set problem". In: *Discrete Applied Mathematics* 126 (2–3), pp. 313–322. DOI: 10.1016/S0166-218X(02)00205-6.

Santini, Alberto (Apr. 2017). "Combinatorial Optimisation Problems in Logistics and Scheduling". PhD thesis. Alma Mater Studiorum Università di Bologna. URL: http://amsdottorato.unibo.it/7908/.

Santini, Alberto (2026). *Instances and Best Known Solutions for Berth Allocation Problems*. DOI: 10.5281/zenodo.24006051. URL: https://github.com/alberto-santini/berth-allocation-problems/.

Santini, Alberto, Henrik Alsing Friberg, and Stefan Ropke (2015). "A note on a model for quay crane scheduling with non-crossing constraints". In: *Engineering Optimization* 47 (6), pp. 860–865. DOI: 10.1080/0305215X.2014.958731.

Vacca, Ilaria, Matteo Salani, and Michel Bierlaire (2012). "An Exact Algorithm for the Integrated Planning of Berth Allocation and Quay Crane Assignment". In: *Transportation Science* 47.2. DOI: 10.1287/trsc.1120.0428.

van der Steeg, Jaap-Jan, Menno Oudshoorn, and Neil Yorke-Smith (2023). "Berth planning and real-time disruption recovery: a simulation study for a tidal port". In: *Flexible Services and Manufacturing Journal* 35.1, pp. 70–110. DOI: 10.1007/s10696-022-09473-8.

Wawrzyniak, Jakub, Maciej Drozdowski, and Éric Sanlaville (2020). "Selecting algorithms for large berth allocation problems". In: *European Journal of Operational Research* 283 (3), pp. 844–862. DOI: 10.1016/j.ejor.2019.11.055.

Zampelli, Stéphane, Yannis Vergados, Rowan Van Schaeren, Wout Dullaert, and Birger Raa (2013). "The Berth Allocation and Quay Crane Assignment Problem Using a CP Approach". In: *Principles and Practice of Constraint Programming* (Sept. 16–20, 2013). Ed. by Christian Schulte. Uppsala, Sweden: Springer Berlin Heidelberg, pp. 880–896. DOI: 10.1007/978-3-642-40627-0_64.

Zhu, Enqiang, Fei Jiang, and Jin Liu Chanjuan amd Xu (2022). "Partition Independent Set and Reduction-Based Approach for Partition Coloring Problem". In: *IEEE Transactions on Cybernetics* 52 (6), pp. 4960–4969. DOI: 10.1109/TCYB.2020.3025819.

A Integer programming models

In this section, we describe the five formulations we use to assess the quality of our proposed approach. We describe these formulations using the notation introduced in this paper and highlight any significant modifications of the original models. Initially, we present each formulation using the "max(compl)" objective function. Then, we explain how to amend each model to solve the cumulative objective functions. In some cases, the original models were already devised for cumulative objective functions and the adaptations we present for the cumulative objective functions mostly undo the edits we made to model the bottleneck function. We use the following sets and parameter throughout the appendix:

$$\mathcal{J}_i = \big\{j \in \{1,\dots,m\} : \delta_j + \ell_i \le L,\ a_i + h_{ij} \le T\big\} \qquad \forall i \in \{1,\dots,n\} \tag{5a}$$
$$\mathcal{T}_{ij} = \{a_i, \dots, T - h_{ij}\} \qquad \forall i \in \{1,\dots,n\},\ \forall j \in \mathcal{J}_i \tag{5b}$$
$$\mathcal{B}_{ij} = \big\{k \in \{1,\dots,m\} : \delta_j \le \delta_k < \delta_j + \ell_i\big\} \qquad \forall i \in \{1,\dots,n\},\ \forall j \in \mathcal{J}_i \tag{5c}$$
$$\underline{h}_i = \min_{j \in \mathcal{J}_i} h_{ij} \qquad \forall i \in \{1,\dots,n\}. \tag{5d}$$

Set $\mathcal{J}_i$ contains the admissible leftmost berths for ship i, $\mathcal{T}_{ij}$ its feasible mooring times when berth j is selected, and $\mathcal{B}_{ij}$ the berths occupied by that assignment.

A.1 Model "RP" of Guan et al. (2004)

This formulation was originally presented for a "hyb|dyn|fix|$\sum$w(wait+hand)" BAP. Furthermore, the authors assume that all berths have the same length. Therefore, the length of a ship is simply expressed as the number of berths it occupies. We denote this parameter as $\hat{\ell}_i$ ($i \in \{1,\dots,n\}$) and will work under the same assumption.

This model uses the following decision variables. Variables $x_{i_1 i_2} \in \{0,1\}$ and $y_{i_1 i_2} \in \{0,1\}$ for each pair of ships i_1, i_2 ($i_1 \neq i_2$). Setting $x_{i_1 i_2} = 1$ enforces that ship i_1 completes before ship i_2 moors, while setting $y_{i_1 i_2} = 1$ enforces that ship i_1 lies entirely to the left of ship i_2. Variable $z_{ij} \in \{0,1\}$, defined for $i \in \{1,\dots,n\}$ and $j \in \mathcal{J}_i$, takes value one if and only if j is the leftmost berth assigned to ship i. Variables $u_i, c_i \in \{1,\dots,T\}$ for each ship i. Variable u_i records the ship's mooring time, and variable c_i denotes the completion time, i.e., the time when the ship departs from the port. Variable $v_i \in \{1,\dots,m\}$ for each ship i, denoting the leftmost berth used by the ship. Variable $\eta_i \ge 0$ is the handling time induced by the selected berth. Variable $C \ge 0$, the total makespan.

The model adapted for a "hyb|dyn|pos|max(compl)" BAP reads as follows.

$$\min \quad C \tag{6a}$$
$$\text{s.t.} \quad C \ge c_i \qquad \forall i \in \{1,\dots,n\} \tag{6b}$$
$$\sum_{j \in \mathcal{J}_i} z_{ij} = 1 \qquad \forall i \in \{1,\dots,n\} \tag{6c}$$
$$v_i = \sum_{j \in \mathcal{J}_i} j z_{ij} \qquad \forall i \in \{1,\dots,n\} \tag{6d}$$
$$\eta_i = \sum_{j \in \mathcal{J}_i} h_{ij} z_{ij} \qquad \forall i \in \{1,\dots,n\} \tag{6e}$$
$$a_i \le u_i \le T - \eta_i \qquad \forall i \in \{1,\dots,n\} \tag{6f}$$
$$c_i = u_i + \eta_i \qquad \forall i \in \{1,\dots,n\} \tag{6g}$$
$$u_{i_2} \ge c_{i_1} - T(1 - x_{i_1 i_2}) \qquad \forall i_1, i_2 \in \{1,\dots,n\},\ i_1 \neq i_2 \tag{6h}$$
$$v_{i_2} \ge v_{i_1} + \hat{\ell}_{i_1} - m(1 - y_{i_1 i_2}) \qquad \forall i_1, i_2 \in \{1,\dots,n\},\ i_1 \neq i_2 \tag{6i}$$
$$x_{i_1 i_2} + x_{i_2 i_1} + y_{i_1 i_2} + y_{i_2 i_1} \ge 1 \qquad \forall i_1, i_2 \in \{1,\dots,n\},\ i_1 \neq i_2 \tag{6j}$$
$$x_{i_1 i_2} + x_{i_2 i_1} \le 1 \qquad \forall i_1, i_2 \in \{1,\dots,n\},\ i_1 \neq i_2 \tag{6k}$$
$$y_{i_1 i_2} + y_{i_2 i_1} \le 1 \qquad \forall i_1, i_2 \in \{1,\dots,n\},\ i_1 \neq i_2. \tag{6l}$$

The objective function (6a), together with constraints (6b), minimises the total makespan. Constraint (6c) selects one admissible berth for each ship. Constraints (6d) and (6e) set the ship position and handling time implied by that choice. Constraints (6f) and (6g) bound the mooring time and set the completion time. Constraints (6h) and (6i) are no-overlap constraints. Constraints (6j), (6k), and (6l) ensure that the correct mutual position of any two ships is identified. Compared with the original formulation, we introduce z and η to model berth-dependent handling times, and C and (6b) to minimise the makespan rather than the weighted sum of completion times.

A.1.1 Cumulative objectives

For a cumulative objective, variable C and constraints (6b) are omitted, and objective (6a) is replaced. Because (6g) gives $c_i - a_i = (u_i - a_i) + \eta_i$, the "$\sum$(compl)" and "$\sum$w(wait+hand)" objectives are, respectively,

$$\min \quad \sum_{i\in\{1,\dots,n\}} c_i \tag{7a}$$

$$\min \quad \sum_{i\in\{1,\dots,n\}} w_i(c_i - a_i). \tag{7b}$$

For the "$\sum$w(wait+tard)" objective, we introduce a non-negative tardiness variable r_i for every ship and impose

$$r_i \geq c_i - s_i \quad \forall i \in \{1,\dots,n\}. \tag{8}$$

The objective is then

$$\min \quad \sum_{i\in\{1,\dots,n\}} w_i\big[(u_i - a_i) + r_i\big]. \tag{9}$$

Constraint (8) and the non-negativity of r_i ensure that there is an optimal solution with $r_i = (c_i - s_i)_+$.

A.2 Model "PA" of Guan et al. (2004)

As mentioned above, Guan et al. (2004) assume that all berths have the same length. While the RP model cannot be straightforwardly modified to overcome this assumption, the PA model can be amended to account for berths of variable lengths.

In the following, we present the more general model that uses the following variables. Variable $x_{ikt} \in \{0,1\}$, defined for every ship i, berth k, and period t, takes value one if and only if ship i occupies berth k during period t. Variable $y_{ij\tau} \in \{0,1\}$, defined for $i \in \{1,\dots,n\}$, $j \in \mathcal{J}_i$, and $\tau \in \mathcal{T}_{ij}$, takes value one if and only if ship i moors at time τ with j as its leftmost berth. Variable $c_i \in \{1,\dots,T\}$ for each ship i records the ships' departure times. The variable $C \geq 0$ is the total makespan. The model reads as follows.

$$\begin{aligned}
\min \quad & C && && (10a)\\
\text{subject to} \quad & C \geq c_i && \forall i \in \{1,\dots,n\} && (10b)\\
& c_i = \sum_{j\in\mathcal{J}_i} \sum_{\tau\in\mathcal{T}_{ij}} (\tau + h_{ij}) y_{ij\tau} && \forall i \in \{1,\dots,n\} && (10c)\\
& \sum_{j\in\mathcal{J}_i} \sum_{\tau\in\mathcal{T}_{ij}} y_{ij\tau} = 1 && \forall i \in \{1,\dots,n\} && (10d)\\
& x_{ikt} = \sum_{\substack{j\in\mathcal{J}_i \\ k\in\mathcal{B}_{ij}}} \sum_{\substack{\tau\in\mathcal{T}_{ij} \\ \tau\leq t<\tau+h_{ij}}} y_{ij\tau} && \forall i \in \{1,\dots,n\},\ \forall k \in \{1,\dots,m\},\ \forall t \in \{1,\dots,T\} && (10e)\\
& \sum_{i\in\{1,\dots,n\}} x_{ikt} \leq 1 && \forall k \in \{1,\dots,m\},\ \forall t \in \{1,\dots,T\}. && (10f)
\end{aligned}$$

The objective function (10a) and constraint (10b) are analogous to (6a) and (6b). Constraint (10c) sets each completion time using the handling time at the selected berth. Constraint (10d) ensures that

each ship is assigned exactly one mooring berth and time. Constraint (10e) links the x and y variables: its right-hand side is one when the selected rectangle covers berth-period pair (k, t). Constraint (10f) ensures that no more than one ship uses each berth at any given time.

A.2.1 Cumulative objectives

For any cumulative objective, variable C and constraint (10b) are omitted. For "$\sum$(compl)" and "$\sum$w(wait+hand)", objective (10a) is replaced by (7a) and (7b), respectively. For "$\sum$w(wait+tard)", we additionally introduce the non-negative variables r_i and constraints (8). Because PA has no explicit mooring-time variable, the waiting time is recovered from the selected y variable and the objective becomes

$$\min \quad \sum_{i\in\{1,\dots,n\}} w_i \left[\sum_{j\in\mathcal{J}_i} \sum_{\tau\in\mathcal{T}_{ij}} (\tau - a_i) y_{ij\tau} + r_i \right]. \tag{11}$$

A.3 Model "GSPP" of Christensen et al. (2008) and Buhrkal et al. (2011)

This model was originally proposed for a "disc|dyn|pos|$\sum$w(wait+hand)" BAP. However, it can be amended to consider a hybrid spatial attribute and to minimise the makespan. To present the model, we use the assignment set V^{BAP} from Section 5 and let $V_i^{\mathrm{BAP}} = \{(i', j, t) \in V^{\mathrm{BAP}} : i' = i\}$. For each berth k and period τ, define

$$V_{k\tau}^{\mathrm{BAP}} = \big\{(i, j, t) \in V^{\mathrm{BAP}} : \delta_j \le \delta_k < \delta_j + \ell_i,\ t \le \tau < t + h_{ij}\big\}.$$

Each set $V_{k\tau}^{\mathrm{BAP}}$ contains all assignments that cause berth k to be occupied in period τ.

The GSPP model uses variables $x_v \in \{0, 1\}$ defined for each $v \in V^{\mathrm{BAP}}$ and a single variable $C \ge 0$. Variable x_v takes the value one for $v = (i, j, t)$ if ship i moors at berth j at time t. The model reads as follows.

$$\begin{aligned}
\min \quad & C && && (12a)\\
\text{subject to} \quad & C \ge (t + h_{ij}) x_v && \forall v = (i, j, t) \in V^{\mathrm{BAP}} && (12b)\\
& \sum_{v\in V_i^{\mathrm{BAP}}} x_v = 1 && \forall i \in \{1, \dots, n\} && (12c)\\
& \sum_{v\in V_{k\tau}^{\mathrm{BAP}}} x_v \le 1 && \forall k \in \{1, \dots, m\},\ \forall \tau \in \{1, \dots, T\}. && (12d)
\end{aligned}$$

Constraint (12c) ensures that each ship is assigned a mooring time and berth. Constraint (12d) prevents more than one ship from using a given berth at a given time.

A.3.1 Cumulative objectives

For a cumulative objective, variable C and constraints (12b) are omitted. Because every coefficient below is determined by the assignment $v = (i, j, t)$, objective (12a) can be replaced directly by one of the following three objectives:

$$\min \quad \sum_{v=(i,j,t)\in V^{\mathrm{BAP}}} (t + h_{ij}) x_v \tag{13a}$$

$$\min \quad \sum_{v=(i,j,t)\in V^{\mathrm{BAP}}} w_i (t - a_i + h_{ij}) x_v \tag{13b}$$

$$\min \quad \sum_{v=(i,j,t)\in V^{\mathrm{BAP}}} w_i \big[(t - a_i) + (t + h_{ij} - s_i)_+\big] x_v. \tag{13c}$$

These objectives model, respectively, "$\sum$(compl)", "$\sum$w(wait+hand)", and "$\sum$w(wait+tard)". In the last objective, tardiness is precomputed for each assignment, so no auxiliary variable is required.

A.4 Model "S" of Ernst et al. (2017)

Ernst et al. (2017) consider a "cont|dyn|fix|$\sum$(compl)" BAP. Indeed, the authors assume the quay is continuous and ships can moor at any point. To adapt this model to a "hyb|dyn|pos|max(compl)" BAP, we assume that the quay is divided into equal-length berths, that ship i occupies $\hat{\ell}_i$ consecutive berths, and that its handling time depends on its leftmost berth.

The model uses the following variables. Variable $x_{i_1 i_2} \in \{0,1\}$ for each pair of ships i_1, i_2 ($i_1 \neq i_2$); setting $x_{i_1 i_2} = 1$ enforces that ship i_1 completes before ship i_2 moors. Variable $I_{i_1 i_2} \in \{0,1\}$ for each pair of ships i_1, i_2 ($i_1 \neq i_2$); setting $I_{i_1 i_2} = 1$ enforces that ship i_1 lies entirely to the left of ship i_2. Variable $z_{ij} \in \{0,1\}$, defined for $i \in \{1,\ldots,n\}$ and $j \in \mathcal{J}_i$; z_{ij} takes value one if and only if j is the leftmost berth assigned to ship i. Variables $u_i, c_i \in \{1,\ldots,T\}$ are the periods when ship i moors and completes operations. Variable u_i is bounded from below by a_i. Variable $y_i \in \{1,\ldots,m\}$ is the leftmost berth used by ship i. Variable $\eta_i \geq 0$ is the handling time induced by the selected berth. $C \geq 0$ is the total makespan.

In the following, we present the model together with those strengthenings of Ernst et al. (2017) that remain valid under berth-dependent handling times.

$$\min \quad C \tag{14a}$$

$$\text{subject to} \quad C \geq c_i \qquad \forall i \in \{1,\ldots,n\} \tag{14b}$$

$$\sum_{j \in \mathcal{J}_i} z_{ij} = 1 \qquad \forall i \in \{1,\ldots,n\} \tag{14c}$$

$$y_i = \sum_{j \in \mathcal{J}_i} j z_{ij} \qquad \forall i \in \{1,\ldots,n\} \tag{14d}$$

$$\eta_i = \sum_{j \in \mathcal{J}_i} h_{ij} z_{ij} \qquad \forall i \in \{1,\ldots,n\} \tag{14e}$$

$$a_i \leq u_i \leq T - \eta_i \qquad \forall i \in \{1,\ldots,n\} \tag{14f}$$

$$c_i = u_i + \eta_i \qquad \forall i \in \{1,\ldots,n\} \tag{14g}$$

$$x_{i_1 i_2} + x_{i_2 i_1} + I_{i_1 i_2} + I_{i_2 i_1} = 1 \qquad \forall i_1, i_2 \in \{1,\ldots,n\},\ i_1 \neq i_2 \tag{14h}$$

$$u_{i_2} \geq c_{i_1} - T(1 - x_{i_1 i_2}) \qquad \forall i_1, i_2 \in \{1,\ldots,n\},\ i_1 \neq i_2 \tag{14i}$$

$$y_{i_2} \geq \hat{\ell}_{i_1} I_{i_1 i_2} + y_{i_1} - (m - \hat{\ell}_{i_1})(1 - I_{i_1 i_2}) \qquad \forall i_1, i_2 \in \{1,\ldots,n\},\ i_1 \neq i_2 \tag{14j}$$

$$\sum_{\substack{i_2 \in \{1,\ldots,n\},\ i_2 \neq i_1 \\ \hat{\ell}_{i_1} + \hat{\ell}_{i_2} > m}} I_{i_1 i_2} = 0 \qquad \forall i_1 \in \{1,\ldots,n\} \tag{14k}$$

$$u_{i_1} \geq \min_{i \in \{1,\ldots,n\}} a_i + \frac{1}{m} \sum_{\substack{i_2 \in \{1,\ldots,n\} \\ i_2 \neq i_1}} \underline{h}_{i_2} \hat{\ell}_{i_2} x_{i_2 i_1} \qquad \forall i_1 \in \{1,\ldots,n\} \tag{14l}$$

$$u_{i_1} \geq \min_{i \in \{1,\ldots,n\}} a_i + \sum_{\substack{i_2 \in \{1,\ldots,n\},\ i_2 \neq i_1 \\ (k+1)\hat{\ell}_{i_2} > m}} \frac{\underline{h}_{i_2}}{k} x_{i_2 i_1} \qquad \forall i_1 \in \{1,\ldots,n\},\ \forall k \in \{1,2,3\}. \tag{14m}$$

The objective function (14a) and constraints (14b) are used to minimise the makespan. By contrast, the original model in (Ernst et al. 2017) minimises the sum of the completion times. Constraints (14c)–(14e) select an admissible berth and set the ship's position and handling time. Constraints (14f) and (14g) bound the mooring time and set the completion time. Constraint (14h) selects exactly one temporal or spatial ordering for each pair of ships. Constraints (14i) and (14j) enforce the selected ordering. The subsequent constraints are valid inequalities adapted from Ernst et al. (2017). Constraint (14k) forbids handling simultaneously a pair of ships that are too long to fit in the quay. Constraint (14l) uses an area bound for ships constrained to complete before i_1 starts, and (14m) uses the fact that at most k ships longer than $m/(k+1)$ berths fit simultaneously. In both inequalities, $\underline{h}_{i_2}$ is a valid lower bound on the handling time of predecessor i_2, irrespective of its eventual berth. We omit

the precedence-based and symmetry-breaking inequalities from the original model. Their exchange and spatial-shift arguments rely on handling times remaining unchanged when ship assignments are swapped or moved along the quay; this is not guaranteed when handling times depend on berth.

A.4.1 Cumulative objectives

For a cumulative objective, variable C and constraints (14b) are omitted. The "$\sum$(compl)" and "$\sum$w(wait+hand)" objectives replace (14a) with, respectively,

$$\min \quad \sum_{i\in\{1,\dots,n\}} c_i \tag{15a}$$

$$\min \quad \sum_{i\in\{1,\dots,n\}} w_i(c_i - a_i). \tag{15b}$$

Objective (15a), together with the remaining constraints stated above, gives the adapted S formulation with the objective for which the model was originally proposed. All feasibility constraints and valid inequalities displayed in the S model remain unchanged for each cumulative objective. For the "$\sum$w(wait+tard)" objective, introduce non-negative variables r_i and impose

$$r_i \geq c_i - s_i \quad \forall i \in \{1,\dots,n\}. \tag{16}$$

The objective is

$$\min \quad \sum_{i\in\{1,\dots,n\}} w_i\big[(u_i - a_i) + r_i\big]. \tag{17}$$

Remark that, even if constraints (16) do not inforce this explicitly, there is an optimal solution with $r_i = (c_i - s_i)_+$ for all $i \in \{1,\dots,n\}$.

A.5 Model "TI" of Ernst et al. (2017)

This model uses the variables and core constraints of model S. It additionally uses $b_{it}, q_{it} \in \{0,1\}$ for $i \in \{1,\dots,n\}$ and $t \in \{1,\dots,T\}$. Variable b_{it} takes value one if and only if ship i has moored by time t, while q_{it} takes value one if and only if it has completed by time t. For simplicity of notation, ler $b_{i0} = q_{i0} = 0$. The model reads as follows.

$$\begin{aligned}
\min \quad & C && && (18a)\\
\text{subject to} \quad & \text{(14b), (14c)–(14j)} && && \\
& b_{i0} = q_{i0} = 0 && \forall i \in \{1,\dots,n\} && (18b)\\
& b_{i,t+1} \geq b_{it}, \quad q_{i,t+1} \geq q_{it} && \forall i \in \{1,\dots,n\},\ \forall t \in \{0,\dots,T-1\} && (18c)\\
& b_{iT} = q_{iT} = 1 && \forall i \in \{1,\dots,n\} && (18d)\\
& q_{it} \leq b_{it} && \forall i \in \{1,\dots,n\},\ \forall t \in \{1,\dots,T\} && (18e)\\
& u_i = \sum_{t=1}^{T} t\big(b_{it} - b_{i,t-1}\big) && \forall i \in \{1,\dots,n\} && (18f)\\
& c_i = \sum_{t=1}^{T} t\big(q_{it} - q_{i,t-1}\big) && \forall i \in \{1,\dots,n\} && (18g)\\
& \sum_{i\in\{1,\dots,n\}} \hat{\ell}_i\big(b_{it} - q_{it}\big) \leq m && \forall t \in \{1,\dots,T\}. && (18h)
\end{aligned}$$

Constraints (18c) and (18d), together with (18b), make each binary sequence switch once from zero to one. Constraints (18f) and (18g) place those switches at the mooring and completion times, respectively. Constraint (18e) strengthens the requirement that completion cannot precede mooring. Consequently, $b_{it} - q_{it}$ equals one in the periods $u_i \leq t < c_i$ during which ship i is handled. Constraint (18h) therefore ensures that the total number of berths occupied by active ships never exceeds m.

A.5.1 Cumulative objectives

The cumulative variants of TI use the same objectives and the same tardiness variables as the S model. Specifically, variable C and the inherited constraints (14b) are omitted, objective (18a) is replaced by (15a), (15b), or (17), and constraints (16) are added for the last objective. All remaining constraints are unchanged.

B Constraint programming models

We use the CP-SAT solver of Perron et al. (2023) to improve the initial solutions, as described in Sections 6.1 and 6.2. To the best of our knowledge, neither of the two models presented below appears in the literature in this exact form. They are, however, partially inspired by previous constraint-based approaches to berth allocation and container-terminal scheduling (Kameshwaran et al. 2013; Rashidi et al. 2013; Zampelli et al. 2013; Qin et al. 2016; van der Steeg et al. 2023). The constraints of the models presented below are common to the bottleneck and cumulative objectives. We first state the models for the "max(compl)" objective and subsequently describe the modifications required for the three cumulative objectives. As in the RP, S, and TI models presented above, we assume that all berths have the same length and denote by $\hat{\ell}_i$ the number of consecutive berths occupied by ship i. This assumption is easy to relax, but it simplifies the notation and the model presentation considerably. We also use set $\mathcal{J}_i$ defined in (5a). Under the assumption that all berths have the same length, its spatial condition is equivalently $j + \hat{\ell}_i - 1 \leq m$. If ship i moors at $j \in \mathcal{J}_i$, it occupies berths $\{j, \dots, j + \hat{\ell}_i - 1\}$. In the models, berth k is represented by the unit interval $[k, k+1)$. For a $\mathrm{DBAP_b}$ call with deadline M, T is replaced by M in the definition of $\mathcal{J}_i$ and in the variable domains; the objective function can be omitted.

B.1 Single-rectangle model

The first model associates one rectangle to each ship. A rectangle is a pair of intervals defining the spatial and temporal location of the ship, as in Figure 3. Variable $v_i \in \mathcal{J}_i$ identifies the berth at which ship i moors, and variables $u_i, c_i \in \{1, \dots, T\}$ denote its mooring and completion times, respectively. The Element constraint uses v_i to select variable $\eta_i = h_{iv_i}$, representing the handling time. Variables $\mathcal{I}_i^{\mathrm{q}}$ and $\mathcal{I}_i^{\mathrm{t}}$ are the quay and temporal intervals associated with ship i, and C is the makespan. The model reads as follows.

$$\begin{aligned}
\min \quad & C && && (19a)\\
\text{subject to} \quad & \eta_i = \mathrm{Element}\big(v_i, (h_{ij})_{j\in\{1,\dots,m\}}\big) && \forall i \in \{1,\dots,n\} && (19b)\\
& \mathcal{I}_i^{\mathrm{q}} = \mathrm{Interval}(v_i, \hat{\ell}_i, v_i + \hat{\ell}_i) && \forall i \in \{1,\dots,n\} && (19c)\\
& \mathcal{I}_i^{\mathrm{t}} = \mathrm{Interval}(u_i, \eta_i, c_i) && \forall i \in \{1,\dots,n\} && (19d)\\
& \mathrm{NoOverlap2D}\big((\mathcal{I}_i^{\mathrm{q}})_{i\in\{1,\dots,n\}}, (\mathcal{I}_i^{\mathrm{t}})_{i\in\{1,\dots,n\}}\big) && && (19e)\\
& C \geq c_i && \forall i \in \{1,\dots,n\}. && (19f)
\end{aligned}$$

Constraint (19b) selects the handling time associated with v_i. An expression $\mathrm{Interval}(s, d, e)$ in (19c) and (19d) represents the half-open interval $[s, e)$ and enforces $s + d = e$. Thus, (19d) also defines $c_i = u_i + \eta_i$, and the two interval constraints create the rectangle $[v_i, v_i + \hat{\ell}_i) \times [u_i, c_i)$. Its spatial dimension corresponds to the berths occupied by ship i, while its temporal dimension corresponds to the periods $u_i, \dots, c_i - 1$ in which the ship occupies the berths. Constraint (19e) requires every pair of rectangles to be disjoint in at least one dimension and, therefore, prevents two ships from occupying the same portion of the quay at the same time. Finally, constraints (19f), together with objective (19a), minimise the completion time of the last ship.

B.2 Optional-rectangle model

The second model uses one optional rectangle for each admissible ship–berth pair. Whereas in the first model the rectangles' coordinates depended on the variables u_i, c_i, v_i, and η_i, here all feasible

$\mathcal{O}(nm)$ spatial positions are precomputed and the rectangles' coordinates only depend on the temporal variables u_i and c_i. Exactly one rectangle is selected for each ship, via binary variables $y_{ij} \in \{0,1\}$, taking the value one if and only if ship i is assigned to berth $j \in \mathcal{J}_i$. The model reads as follows.

$$\begin{aligned}
\min \quad & C && && \text{(20a)}\\
\text{subject to} \quad & \sum_{j \in \mathcal{J}_i} y_{ij} = 1 && \forall i \in \{1,\dots,n\} && \text{(20b)}\\
& \mathcal{I}^{\mathrm{q}}_{ij} = \text{OptionalInterval}(j, \hat{\ell}_i, j+\hat{\ell}_i; y_{ij}) && \forall i \in \{1,\dots,n\},\ j \in \mathcal{J}_i && \text{(20c)}\\
& \mathcal{I}^{\mathrm{t}}_{ij} = \text{OptionalInterval}(u_i, h_{ij}, c_i; y_{ij}) && \forall i \in \{1,\dots,n\},\ j \in \mathcal{J}_i && \text{(20d)}\\
& \text{NoOverlap2D}\big((\mathcal{I}^{\mathrm{q}}_{ij})_{i\in\{1,\dots,n\},\ j\in\mathcal{J}_i}, (\mathcal{I}^{\mathrm{t}}_{ij})_{i\in\{1,\dots,n\},\ j\in\mathcal{J}_i}\big) && && \text{(20e)}\\
& C \geq c_i && \forall i \in \{1,\dots,n\}. && \text{(20f)}
\end{aligned}$$

Constraint (20b) assigns one berth to each ship. The optional intervals in (20c) and (20d) are absent and ignored when $y_{ij} = 0$; when $y_{ij} = 1$, they represent mooring spatial/temporal intervals and enforce that their start plus size equals their end. Consequently, the active intervals form the rectangle $[j, j+\hat{\ell}_i) \times [u_i, c_i)$ and impose $c_i = u_i + h_{ij}$ for the selected berth. Both intervals of a pair are associated to the same binary variable y_{ij}, so (20e) considers exactly one rectangle per ship.

B.3 Cumulative objectives

For the BAP_{c}, constraints (19f) or (20f) and the corresponding makespan objective are omitted. Objective functions "$\sum$(compl)" and "$\sum$w(wait+hand)" are modelled, respectively, using the following objectives:

$$\min \quad \sum_{i\in\{1,\dots,n\}} c_i \tag{21a}$$

$$\min \quad \sum_{i\in\{1,\dots,n\}} w_i(c_i - a_i). \tag{21b}$$

For the "$\sum$w(wait+tard)" objective, we additionally introduce a tardiness variable r_i for each ship and impose the following constraint

$$r_i = \max\{0, c_i - s_i\} \quad \forall i \in \{1,\dots,n\}. \tag{22}$$

The objective is then

$$\min \quad \sum_{i\in\{1,\dots,n\}} w_i\big[(u_i - a_i) + r_i\big]. \tag{23}$$